

# Gestión de Aplicaciones Web Empresariales

Práctica Experimental — Unidad II

## Práctica Experimental Unidad II

Aplicaciones Web Dinámicas con Autenticación,  
Seguridad OWASP y Múltiples Tecnologías de Servidor

### Equipo de Desarrollo

| # | Nombre                       | Rol        |
|---|------------------------------|------------|
| 1 | BELTRÁN MONTIEL FRED ADRIÁN  | Integrante |
| 2 | CRUZ PÉREZ JUSTYN KEITH      | Integrante |
| 3 | PALLO PINTO DANIEL ALEJANDRO | Integrante |

**Fecha:** 20 de junio de 2026

**Proyecto:** MiPrimerMVC (ASP.NET Core 8 + PostgreSQL)

**Tecnologías:** ASP.NET Core 8 MVC & PHP

**Repositorio:**  <https://github.com/keithdrox/practica-aspnet>

Entrega correspondiente al período académico 2026 — Unidad II

## Índice

---

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                         | <b>3</b>  |
| <b>2. Directriz (1): Revisión Bibliográfica</b>                                | <b>3</b>  |
| <b>3. Fundamento Teórico</b>                                                   | <b>3</b>  |
| 3.1. Patrón Arquitectónico MVC . . . . .                                       | 4         |
| 3.2. Entity Framework Core y Consultas Parametrizadas . . . . .                | 4         |
| 3.3. Autenticación por Cookie en ASP.NET Core . . . . .                        | 4         |
| 3.4. OWASP Top 10 — Controles Relevantes . . . . .                             | 4         |
| <b>4. Directriz (2) y (3): Aplicación Web en ASP.NET Core</b>                  | <b>5</b>  |
| 4.1. Arquitectura del Proyecto . . . . .                                       | 5         |
| 4.2. Base de Datos PostgreSQL y Migraciones . . . . .                          | 5         |
| 4.3. Modelo de Datos . . . . .                                                 | 6         |
| 4.3.1. Modelo Producto . . . . .                                               | 6         |
| 4.3.2. Modelo Usuario . . . . .                                                | 6         |
| 4.4. Módulo CRUD — ProductoController . . . . .                                | 7         |
| 4.5. Sistema de Autenticación . . . . .                                        | 8         |
| 4.5.1. Registro de Usuarios — AuthController . . . . .                         | 8         |
| 4.5.2. Login y Creación de Cookie de Sesión . . . . .                          | 9         |
| <b>5. Directriz (3): Controles de Seguridad OWASP</b>                          | <b>10</b> |
| 5.1. Ciclo de Vida del Servidor Web y Arquitecturas de Integración . . . . .   | 10        |
| 5.1.1. Modelos de integración servidor-lenguaje . . . . .                      | 10        |
| 5.1.2. PERL/CGI: contexto histórico y limitaciones actuales . . . . .          | 11        |
| 5.1.3. PHP 8.x: características clave para el desarrollo web moderno . . . . . | 11        |
| 5.1.4. Criterios de selección de tecnología de servidor . . . . .              | 12        |
| 5.2. Autenticación Segura y Gestión de Sesiones . . . . .                      | 13        |
| 5.2.1. Algoritmos de hash de contraseñas: bcrypt frente a Argon2id . . . . .   | 13        |
| 5.2.2. Session Fixation y regeneración del identificador de sesión . . . . .   | 13        |
| 5.2.3. JSON Web Tokens (JWT) como alternativa a sesiones con estado . . . . .  | 14        |
| 5.2.4. Síntesis: controles aplicados frente a OWASP A01/A02/A07 . . . . .      | 15        |
| 5.3. (b) Token CSRF en todos los formularios . . . . .                         | 15        |
| 5.4. Pipeline de Middleware en ASP.NET Core . . . . .                          | 16        |
| 5.5. Inyección de Dependencias (DI) en ASP.NET Core . . . . .                  | 17        |
| 5.6. Spring Boot: IoC Container y Spring Security . . . . .                    | 18        |
| 5.7. Principios SOLID Aplicados al Backend del PFC . . . . .                   | 20        |
| 5.7.1. S — Principio de Responsabilidad Única (SRP) . . . . .                  | 20        |
| 5.7.2. O — Principio Abierto/Cerrado (OCP) . . . . .                           | 21        |
| 5.7.3. L — Principio de Sustitución de Liskov (LSP) . . . . .                  | 21        |
| 5.7.4. I — Principio de Segregación de Interfaces (ISP) . . . . .              | 21        |
| 5.7.5. D — Principio de Inversión de Dependencias (DIP) . . . . .              | 22        |
| 5.8. Inyección SQL: Mecanismo, Consecuencias y Severidad CVSS . . . . .        | 22        |
| 5.8.1. bindParam() vs bindValue() en PDO . . . . .                             | 23        |
| 5.8.2. Comparativa ADO.NET (ASP.NET) vs JDBC (Java) . . . . .                  | 24        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>6. Anexo C: Preguntas de Análisis</b>                            | <b>26</b> |
| <b>7. Anexo D: Architecture Decision Records (ADR)</b>              | <b>28</b> |
| 7.1. ADR-001: Selección de Tecnología de Backend . . . . .          | 28        |
| 7.2. ADR-002: Estrategia de Base de Datos . . . . .                 | 28        |
| <b>8. Directriz (4): Tabla Comparativa de Tecnologías</b>           | <b>29</b> |
| <b>9. Directriz (2): Aplicación en Segunda Tecnología (PHP)</b>     | <b>31</b> |
| 9.1. Configuración del entorno PHP . . . . .                        | 32        |
| 9.2. Conexión PDO con prepared statements . . . . .                 | 32        |
| 9.3. Patrón Repository en PHP . . . . .                             | 33        |
| 9.3.1. Interfaz ProductoRepositoryInterface . . . . .               | 33        |
| 9.3.2. Clase concreta PdoProductoRepository . . . . .               | 33        |
| 9.4. Análisis Estático con PHPStan . . . . .                        | 35        |
| 9.5. Análisis Estático en ASP.NET Core (Roslyn Analyzers) . . . . . | 35        |
| 9.6. DDL de la base de datos PostgreSQL . . . . .                   | 36        |
| 9.7. Sistema de autenticación PHP . . . . .                         | 37        |
| 9.7.1. Registro de usuario . . . . .                                | 37        |
| 9.7.2. Login y creación de sesión . . . . .                         | 38        |
| 9.7.3. Logout . . . . .                                             | 38        |
| 9.8. Guard de sesión reutilizable . . . . .                         | 39        |
| 9.9. Módulo CRUD de Productos . . . . .                             | 39        |
| 9.9.1. Listado de productos . . . . .                               | 39        |
| 9.9.2. Creación de producto con token CSRF . . . . .                | 39        |
| 9.9.3. Eliminación de producto . . . . .                            | 40        |
| 9.10. Resumen de controles OWASP en la aplicación PHP . . . . .     | 40        |
| <b>10.Resultados y Capturas del Sistema ASP.NET Core</b>            | <b>40</b> |
| 10.1. Pantalla de Login . . . . .                                   | 41        |
| 10.2. Pantalla de Registro . . . . .                                | 41        |
| 10.3. Catálogo de Productos (ruta protegida) . . . . .              | 42        |
| 10.4. Formulario de Creación de Producto . . . . .                  | 42        |
| 10.5. Confirmación de Eliminación . . . . .                         | 43        |
| 10.6. Verificación de Cabeceras HTTP de Seguridad . . . . .         | 43        |
| 10.7. Historial de Commits del Repositorio . . . . .                | 44        |
| <b>11.Conclusiones</b>                                              | <b>44</b> |
| <b>12.Referencias</b>                                               | <b>45</b> |

## 1. Introducción

---

La presente práctica experimental tiene por objetivo investigar y desarrollar, con rigor científico-bibliográfico, los contenidos de la Unidad II (Temas 5–7) referentes al desarrollo de aplicaciones web dinámicas seguras con múltiples tecnologías de servidor. El equipo implementa un sistema de gestión de productos completo en **ASP.NET Core 8 MVC** conectado a **PostgreSQL**, incluyendo autenticación de usuarios, módulo CRUD con consultas parametrizadas y los controles de seguridad obligatorios del estándar **OWASP Top 10**.

## 2. Directriz (1): Revisión Bibliográfica

---

A continuación se presentan las referencias bibliográficas utilizadas como base teórica para el desarrollo de esta práctica:

1. Shahid, J., Hameed, M. K., Javed, I. T., Qureshi, K. N., Ali, M., & Crespi, N. (2022). A comparative study of web application security parameters: Current trends and future directions. *Applied Sciences*, 12(8), 4077. <https://doi.org/10.3390/app12084077>
2. Yin, Z., & Lee, S. U.-J. (2023). Security analysis of web open-source projects based on Java and PHP. *Electronics*, 12(12), 2618. <https://doi.org/10.3390/electronics12122618>
3. Suchanowski, J., & Plechawska-Wójcik, M. (2023). Performance analysis of web applications created in the Spring and Laravel frameworks. *Journal of Computer Sciences Institute*, 29, 304–311. <https://doi.org/10.35784/jcsi.3770>
4. Truszkowski, K., & Pańczyk, M. (2022). Comparative analysis of web application development on Java and PHP. *Journal of Computer Sciences Institute*, 25, 379–383. <https://doi.org/10.35784/jcsi.3050>
5. Kaźmierak, I. (2025). Comparison of the effectiveness of tools for testing the security of web applications. *Journal of Computer Sciences Institute*, 34. <https://doi.org/10.35784/jcsi.6613>
6. Microsoft. (2024). *ASP.NET Core security topics*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/security/>
7. OWASP Foundation. (2021). *OWASP Top Ten*. Open Web Application Security Project. <https://owasp.org/www-project-top-ten/>
8. Microsoft. (2024). *Entity Framework Core — Getting Started*. <https://learn.microsoft.com/en-us/ef/core/get-started/overview/first-app>

## 3. Fundamento Teórico

---

### 3.1. Patrón Arquitectónico MVC

ASP.NET Core MVC implementa el patrón **Modelo-Vista-Controlador (MVC)**, que separa una aplicación en tres componentes:

- **Modelo (Model):** Representa los datos y las reglas de negocio. Utiliza *Data Annotations* para validaciones declarativas.
- **Vista (View):** Motor de plantillas **Razor** (`.cshtml`) que genera HTML dinámico. Codifica salidas automáticamente para prevenir XSS.
- **Controlador (Controller):** Gestiona solicitudes HTTP, coordina Modelo y Vista, y aplica lógica de seguridad.

### 3.2. Entity Framework Core y Consultas Parametrizadas

**Entity Framework Core (EF Core)** es el ORM oficial de Microsoft para .NET. Proporciona acceso a bases de datos relacionales a través de objetos C#. Toda consulta generada por LINQ/EF Core utiliza internamente **consultas parametrizadas**, previniendo inyección SQL de forma transparente. Esto es funcionalmente equivalente a los *prepared statements* de PHP o JDBC.

```
1 // EF Core genera internamente: SELECT * FROM "Productos" WHERE "Id" =  
  $1  
2 var producto = await _context.Productos  
3   .FirstOrDefaultAsync(p => p.Id == id);
```

Listing 1: EF Core usa parámetros automáticamente — equivalente a prepared statements

### 3.3. Autenticación por Cookie en ASP.NET Core

El sistema de autenticación implementado utiliza **cookies cifradas con HttpOnly y SameSite=Strict**, gestionadas por el middleware `CookieAuthenticationDefaults`. Los **Claims** del usuario (nombre, email, ID) se almacenan en la cookie de sesión, y la protección de rutas se implementa con el atributo `[Authorize]`.

### 3.4. OWASP Top 10 — Controles Relevantes

El **OWASP Top 10** es el estándar de referencia para la seguridad en aplicaciones web. Los controles implementados en esta práctica abordan directamente:

- **A03 — Inyección (SQL Injection):** Mitigado por EF Core con consultas parametrizadas.
- **A01 — Control de Acceso Roto:** Mitigado con `[Authorize]` en rutas protegidas.
- **A03 — XSS:** Mitigado por el motor Razor con codificación de salida automática.

- **A01 — CSRF:** Mitigado con [ValidateAntiForgeryToken] en todos los formularios POST.
- **A02 — Fallas Criptográficas:** Mitigado con PasswordHasher<Usuario> (PBKDF2-HMAC-SHA256, sal aleatoria de 128 bits, 100 000 iteraciones).

## 4. Directriz (2) y (3): Aplicación Web en ASP.NET Core

Desarrollado por: *CRUZ PÉREZ JUSTYN KEITH Y COMPAÑEROS*

### 4.1. Arquitectura del Proyecto

El proyecto MiPrimerMVC está construido sobre **ASP.NET Core 8.0 MVC** con conexión a **PostgreSQL 16** mediante **Entity Framework Core 8**. La estructura de archivos principal es:

Cuadro 1: Estructura del proyecto MiPrimerMVC

| Archivo / Carpeta                 | Propósito                                          |
|-----------------------------------|----------------------------------------------------|
| Controllers/AuthController.cs     | , Login y Logout de usuarios                       |
| Controllers/ProductoController.cs | CRUD de productos (protegido con [Authorize])      |
| Models/Usuario.cs                 | Entidad de usuario con PasswordHash                |
| Models/Producto.cs                | Entidad de producto con Data Annotations           |
| Data/ApplicationDbContext.cs      | Contexto de EF Core (sesión con PostgreSQL)        |
| Views/Auth/                       | Vistas de Login y Registro                         |
| Views/Producto/                   | Vistas de listado, creación y eliminación          |
| Migrations/                       | Migraciones de base de datos generadas por EF Core |
| Program.cs                        | Configuración de servicios y pipeline HTTP         |
| appsettings.json                  | Cadena de conexión a PostgreSQL                    |

### 4.2. Base de Datos PostgreSQL y Migraciones

La conexión a PostgreSQL se configuró mediante la cadena de conexión en `appsettings.json` y el proveedor `Npgsql.EntityFrameworkCore.PostgreSQL`:

```

1 {
2   "ConnectionStrings": {
3     "DefaultConnection":
4       "Host=localhost;Database=mi_tienda;Username=postgres;Password
        =*****"
5   }
6 }
```

Listing 2: appsettings.json — Cadena de conexión a PostgreSQL

Se ejecutaron dos migraciones que crearon las tablas en PostgreSQL:

```
dotnet ef migrations add InitialCreate      # Crea tabla Productos
dotnet ef migrations add AddUsuariosTable   # Crea tabla Usuarios
dotnet ef database update                   # Aplica cambios a
PostgreSQL
```

Listing 3: Comandos de migración ejecutados

```
1 CREATE TABLE "Usuarios" (
2     "Id" integer GENERATED BY DEFAULT AS IDENTITY,
3     "NombreUsuario" character varying(50) NOT NULL,
4     "Email" character varying(100) NOT NULL,
5     "PasswordHash" text NOT NULL,
6     "FechaRegistro" timestamp with time zone NOT NULL,
7     CONSTRAINT "PK_Usuarios" PRIMARY KEY ("Id")
8 );
9 CREATE UNIQUE INDEX "IX_Usuarios_Email" ON "Usuarios" ("Email");
```

Listing 4: DDL generado por EF Core para la tabla Usuarios

### 4.3. Modelo de Datos

#### 4.3.1. Modelo Producto

```
1 public class Producto
2 {
3     [Key]
4     public int Id { get; set; }
5
6     [Required(ErrorMessage = "El nombre del producto es obligatorio.")]
7     [StringLength(100, MinimumLength = 3)]
8     [Display(Name = "Nombre del Producto")]
9     public string Nombre { get; set; } = string.Empty;
10
11     [Required(ErrorMessage = "El precio es obligatorio.")]
12     [Range(0.01, 10000.00)]
13     [DataType(DataType.Currency)]
14     public decimal Precio { get; set; }
15
16     [StringLength(250)]
17     public string? Descripcion { get; set; }
18
19     [Required][Range(0, 1000)]
20     [Display(Name = "Cantidad en Stock")]
21     public int Stock { get; set; }
22 }
```

Listing 5: Models/Producto.cs

#### 4.3.2. Modelo Usuario

```
1 public class Usuario
2 {
3     [Key]
4     public int Id { get; set; }
5
6     [Required][StringLength(50)]
7     public string NombreUsuario { get; set; } = string.Empty;
8
9     [Required][StringLength(100)]
10    public string Email { get; set; } = string.Empty;
11
12    // OWASP (c): Almacenado con PasswordHasher<T>      PBKDF2-HMAC-SHA256
13    [Required]
14    public string PasswordHash { get; set; } = string.Empty;
15
16    public DateTime FechaRegistro { get; set; } = DateTime.UtcNow;
17 }
```

Listing 6: Models/Usuario.cs — Con campo PasswordHash (OWASP c)

#### 4.4. Módulo CRUD — ProductoController

El controlador está protegido con [Authorize], asegurando que solo usuarios autenticados puedan acceder. EF Core utiliza consultas parametrizadas automáticamente:

```
1 [Authorize] // Protección de rutas (OWASP A01)
2 public class ProductoController : Controller
3 {
4     private readonly ApplicationDbContext _context;
5
6     public ProductoController(ApplicationDbContext context)
7     => _context = context;
8
9     // GET: /Producto/Index
10    public async Task<IActionResult> Index()
11    {
12        // EF Core usa consultas parametrizadas internamente
13        var productos = await _context.Productos.ToListAsync();
14        return View(productos);
15    }
16
17    // POST: /Producto/Create
18    [HttpPost]
19    [ValidateAntiForgeryToken] // OWASP (b): Token CSRF
20    public async Task<IActionResult> Create(Producto producto)
21    {
22        if (ModelState.IsValid)
23        {
24            _context.Add(producto);
25            await _context.SaveChangesAsync();
26            return RedirectToAction(nameof(Index));
27        }
28        return View(producto);
29    }
30 }
```



```
31 // POST: /Producto/Delete/5
32 [HttpPost, ActionName("Delete")]
33 [ValidateAntiForgeryToken] // OWASP (b): Token CSRF
34 public async Task<IActionResult> DeleteConfirmed(int id)
35 {
36     var producto = await _context.Productos.FindAsync(id);
37     if (producto != null)
38     {
39         _context.Productos.Remove(producto);
40         await _context.SaveChangesAsync();
41     }
42     return RedirectToAction(nameof(Index));
43 }
44 }
```

Listing 7: Controllers/ProductoController.cs — CRUD con EF Core

## 4.5. Sistema de Autenticación

### 4.5.1. Registro de Usuarios — AuthController

```
1 // Inyectar PasswordHasher en el constructor
2 private readonly ApplicationDbContext _context;
3 private readonly PasswordHasher<Usuario> _hasher;
4
5 public AuthController(ApplicationDbContext context)
6 {
7     _context = context;
8     _hasher = new PasswordHasher<Usuario>();
9 }
10
11 [HttpPost]
12 [ValidateAntiForgeryToken] // OWASP (b)
13 public async Task<IActionResult> Register(RegisterViewModel model)
14 {
15     if (!ModelState.IsValid) return View(model);
16
17     bool emailExists = await _context.Usuarios
18         .AnyAsync(u => u.Email == model.Email);
19     if (emailExists)
20     {
21         ModelState.AddModelError("Email",
22             "Este correo ya est registrado.");
23         return View(model);
24     }
25
26     var usuario = new Usuario
27     {
28         NombreUsuario = model.NombreUsuario,
29         Email = model.Email,
30         FechaRegistro = DateTime.UtcNow
31     };
32
33     // OWASP A02: PBKDF2-HMAC-SHA256, sal aleatoria 128 bits,
34     // ~100 000 iteraciones. Sin sal fija en el codigo fuente.
```

```
35     usuario.PasswordHash = _hasher.HashPassword(usuario, model.  
36         Password);  
37     _context.Usuarios.Add(usuario);  
38     await _context.SaveChangesAsync();  
39     return RedirectToAction(nameof(Login));  
40 }
```

Listing 8: AuthController.cs — Registro con PasswordHasher&lt;T&gt;(OWASP A02)

#### 4.5.2. Login y Creación de Cookie de Sesión

```
1 [HttpPost]  
2 [ValidateAntiForgeryToken] // OWASP (b)  
3 public async Task<IActionResult> Login(LoginViewModel model,  
4     string? returnUrl = null)  
5 {  
6     if (!ModelState.IsValid) return View(model);  
7  
8     var usuario = await _context.Usuarios  
9         .FirstOrDefaultAsync(u => u.Email == model.Email);  
10  
11     if (usuario == null ||  
12         usuario.PasswordHash != HashPassword(model.Password))  
13     {  
14         ModelState.AddModelError(string.Empty,  
15             "Correo o contrase a incorrectos.");  
16         return View(model);  
17     }  
18  
19     var claims = new List<Claim>  
20     {  
21         new Claim(ClaimTypes.NameIdentifier, usuario.Id.ToString()),  
22         new Claim(ClaimTypes.Name, usuario.NombreUsuario),  
23         new Claim(ClaimTypes.Email, usuario.Email)  
24     };  
25  
26     var identity = new ClaimsIdentity(claims,  
27         CookieAuthenticationDefaults.AuthenticationScheme);  
28     var properties = new AuthenticationProperties  
29     {  
30         IsPersistent = model.RememberMe,  
31         ExpiresUtc = model.RememberMe  
32             ? DateTimeOffset.UtcNow.AddDays(7)  
33             : DateTimeOffset.UtcNow.AddHours(2)  
34     };  
35  
36     await HttpContext.SignInAsync(  
37         CookieAuthenticationDefaults.AuthenticationScheme,  
38         new ClaimsPrincipal(identity),  
39         properties);  
40  
41     return RedirectToAction("Index", "Producto");  
42 }
```

Listing 9: AuthController.cs — Login con cookie de autenticación

## 5. Directriz (3): Controles de Seguridad OWASP

---

Desarrollado por: **CRUZ PÉREZ JUSTYN KEITH**

### 5.1. Ciclo de Vida del Servidor Web y Arquitecturas de Integración

El servidor web actúa como intermediario entre el cliente HTTP y el código de servidor. La forma en que ese código se integra con el servidor ha evolucionado radicalmente desde los años noventa, con implicaciones directas en rendimiento, consumo de memoria y escalabilidad.

#### 5.1.1. Modelos de integración servidor-lenguaje

**CGI clásico (Common Gateway Interface)** fue el primer mecanismo estándar de integración, definido en RFC 3875. Ante cada solicitud HTTP el servidor crea un *proceso hijo* independiente, ejecuta el script, captura su salida estándar (stdout) y destruye el proceso al terminar. Este modelo es intrínsecamente sin estado: no hay memoria compartida entre solicitudes. Su principal limitación es el coste de creación de procesos: en pruebas de carga con Apache 2.4 y CGI puro, el servidor procesa entre 50 y 150 solicitudes por segundo (rps) antes de saturarse, frente a miles con modelos modernos [10].

**Módulos del servidor (mod\_php)** resuelven el problema de CGI incrustando el intérprete PHP directamente en el proceso de Apache. El intérprete se inicializa una sola vez y persiste en memoria durante todo el ciclo de vida del servidor. Esto elimina el coste de creación de procesos, pero acopla el intérprete al proceso padre de Apache: un fallo del intérprete puede derribar el servidor completo. Además, en configuraciones con **mpm\_prefork**, cada proceso hijo lleva su propia copia del intérprete en memoria, lo que puede consumir varios centenares de MB con tráfico alto.

**FastCGI / PHP-FPM** (PHP FastCGI Process Manager) separa el intérprete PHP en un pool de procesos independientes que se comunican con el servidor web mediante un socket Unix o TCP. Esto aporta tres ventajas clave: (1) los procesos PHP se gestionan independientemente del servidor web, permitiendo reinicios sin downtime; (2) el pool es configurable (**pm.max\_children**, **pm.start\_servers**), adaptándose a la carga; (3) el servidor web puede ser Nginx, Caddy o cualquier otro, no solo Apache. PHP-FPM con Nginx alcanza típicamente entre 1 000 y 5 000 rps en benchmarks de página dinámica simple, dependiendo del hardware [13].

**Servidores embebidos** son la arquitectura dominante en plataformas modernas. **Kestrel** (ASP.NET Core) es un servidor HTTP multiplataforma escrito en C# sobre libuv/sockets asíncronos; se integra directamente en el proceso de la aplicación y puede situarse detrás de un proxy inverso (Nginx, IIS) o exponerse directamente. Los benchmarks TechEmpower Framework ubican a ASP.NET Core con Kestrel consistentemente entre los cinco primeros en la categoría *Plaintext*, superando los 500 000 rps en hardware de referencia [11]. **Tomcat embebido** en Spring Boot sigue un modelo similar: el servidor Tomcat se empaqueta dentro del JAR de la aplicación y arranca con ella, eliminando la necesidad de desplegar un WAR en un servidor externo.

Cuadro 2: Comparativa de modelos de integración servidor-lenguaje

| Modelo            | Proceso/solicitud | Memoria           | RPS aprox. | Aislamiento |
|-------------------|-------------------|-------------------|------------|-------------|
| CGI clásico       | Nuevo proceso     | Baja por proceso  | 50–150     | Alto        |
| mod_php (prefork) | Compartido        | Alta (por worker) | 500–1500   | Bajo        |
| FastCGI/PHP-FPM   | Pool separado     | Media (pool fijo) | 1000–5000  | Medio       |
| Kestrel/ASP.NET   | Asíncrono         | Baja (async I/O)  | 50 000+    | Alto        |
| Tomcat embebido   | Thread pool       | Media             | 10 000+    | Alto        |

### 5.1.2. PERL/CGI: contexto histórico y limitaciones actuales

Perl fue durante los años noventa el lenguaje de facto para scripting web bajo CGI. Su módulo `CGI.pm` abstraía el protocolo CGI: leía variables de entorno (`REQUEST_METHOD`, `QUERY_STRING`, `CONTENT_LENGTH`), parseaba `stdin` para datos POST y generaba las cabeceras HTTP de respuesta. La popularidad de Perl decayó conforme PHP, Python y Ruby ofrecían mejores abstracciones para desarrollo web y frameworks MVC.

En 2025 Perl sigue siendo relevante en tres nichos bien definidos: (1) la **administración de sistemas**, donde herramientas como `cpanm`, scripts de procesamiento de logs y utilidades de red están escritas en Perl; (2) la **bioinformática**, donde BioPerl sigue siendo la librería de referencia para manipulación de secuencias genómicas; y (3) el **scripting de texto**, aprovechando las expresiones regulares de Perl, que siguen siendo las más potentes entre los lenguajes de propósito general. Para desarrollo web nuevo, sin embargo, Perl/CGI no es una opción competitiva frente a PHP-FPM, Node.js o ASP.NET Core [10].

### 5.1.3. PHP 8.x: características clave para el desarrollo web moderno

PHP 8.0 introdujo el **compilador JIT** (*Just-In-Time*), que compila bytecode a código máquina nativo en tiempo de ejecución. Aunque el beneficio es mayor en cargas de cómputo intensivo que en cargas I/O-bound típicas de aplicaciones web, los benchmarks de Phoronix muestran mejoras de entre el 10 % y el 45 % en operaciones matemáticas respecto a PHP 7.4. Las demás mejoras relevantes para este proyecto son:

- **Match expressions** (PHP 8.0): alternativa tipada y sin *fall-through* al `switch`, que reduce errores de lógica.
- **Union types** (PHP 8.0): permiten declarar `int|string` como tipo de parámetro, mejorando la detección estática de errores con PHPStan.
- **Named arguments** (PHP 8.0): permiten llamar funciones especificando el nombre del parámetro, mejorando la legibilidad.
- **Fibers** (PHP 8.1): primitiva de concurrencia cooperativa similar a las corrutinas, base de frameworks asíncronos como ReactPHP.
- **Readonly classes** (PHP 8.2): todas las propiedades son inmutables tras la construcción, útil para DTOs y objetos de valor.

- **Enums tipados** (PHP 8.1): enumeraciones nativas que reemplazan las constantes de clase, con soporte para métodos e interfaces.

En benchmarks comparativos publicados por Kinsta (2023), PHP 8.2 ejecuta el mismo conjunto de operaciones web un **35 % más rápido** que PHP 7.4 y un **8 % más rápido** que PHP 8.1, confirmando que la migración a PHP 8.x tiene impacto medible en producción [12].

#### 5.1.4. Criterios de selección de tecnología de servidor

La elección de tecnología de servidor para el Proyecto Fin de Curso debe basarse en criterios técnicos objetivos y en el contexto del mercado local. La siguiente tabla presenta diez criterios de decisión para PHP 8.x, ASP.NET Core 8 y Java/Spring Boot 3:

| Criterio                      | PHP 8.x                                                                                   | ASP.NET Core 8                                     | Java/Spring Boot 3                            |
|-------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------|-----------------------------------------------|
| Curva de aprendizaje          | Baja: sintaxis permisiva, documentación extensa en español                                | Media: requiere C# y conceptos .NET                | Alta: ecosistema complejo, verbosidad Java    |
| Hosting en Ecuador            | Muy alto: LAMP disponible en todo proveedor local desde \$3/mes                           | Medio: requiere VPS con .NET runtime (\$10–20/mes) | Bajo: VPS con JDK 21, mayor coste operativo   |
| Comunidad                     | Muy grande: 77 % de sitios web con lenguaje de servidor conocido usan PHP (W3Techs, 2024) | Grande: respaldo de Microsoft, activa en GitHub    | Grande: ecosistema empresarial maduro         |
| Ecosistema de paquetes        | Packagist/Composer: 400 000+ paquetes                                                     | NuGet: 400 000+ paquetes                           | Maven Central: 500 000+ artefactos            |
| Rendimiento (RPS)             | 1 000–5 000 con PHP-FPM + Nginx                                                           | 50 000+ con Kestrel (TechEmpower)                  | 10 000–30 000 con Tomcat embebido             |
| Salario desarrollador Ecuador | \$800–1 500/mes (oferta alta)                                                             | \$1 200–2 000/mes (oferta media)                   | \$1 500–2 500/mes (oferta baja)               |
| Integración con cloud         | Buena: AWS Elastic Beanstalk, Google Cloud Run                                            | Excelente: Azure App Service optimizado            | Buena: AWS Elastic Beanstalk, GKE             |
| Licenciamiento                | PHP License (open source, gratuito)                                                       | MIT License (.NET open source desde 2014)          | Apache License 2.0 (open source)              |
| Soporte LTS                   | PHP 8.3: soporte activo hasta nov. 2027                                                   | .NET 8 LTS: soporte hasta nov. 2026                | Spring Boot 3.x + JDK 21 LTS hasta sept. 2029 |

| Criterio              | PHP 8.x                                           | ASP.NET Core 8                                     | Java/Spring Boot 3                                         |
|-----------------------|---------------------------------------------------|----------------------------------------------------|------------------------------------------------------------|
| Seguridad por defecto | Media: CSRF y XSS requieren implementación manual | Alta: CSRF, XSS y HTTPS integrados en el framework | Alta: Spring Security incluye CSRF y autenticación robusta |

## 5.2. Autenticación Segura y Gestión de Sesiones

### 5.2.1. Algoritmos de hash de contraseñas: bcrypt frente a Argon2id

La categoría **A02 — Fallas Criptográficas** del OWASP Top 10 exige que las credenciales nunca se almacenen en texto plano ni mediante funciones de hash rápidas de propósito general (MD5, SHA-1, SHA-256 sin factor de trabajo) [2]. El proyecto PHP de esta práctica utiliza `password_hash()` con el algoritmo **bcrypt** (factor de coste 12) [7], una función basada en el cifrado Blowfish que incorpora un factor de trabajo configurable: cada incremento del coste duplica el tiempo de cómputo necesario, permitiendo ajustar la resistencia frente a fuerza bruta a medida que el hardware se vuelve más potente.

Sin embargo, bcrypt presenta dos limitaciones documentadas: solo procesa los primeros 72 bytes de la contraseña de entrada, y su coste computacional depende exclusivamente de ciclos de CPU, por lo que es vulnerable a paralelización masiva en GPU o hardware especializado (FPGA/ASIC). **Argon2id**, ganador de la Password Hashing Competition de 2015 y recomendado actualmente por OWASP como primera opción para almacenamiento de contraseñas, resuelve esta debilidad al ser una función *memory-hard*: además del parámetro de tiempo, exige una cantidad configurable de memoria RAM por intento de cómputo, encareciendo desproporcionadamente los ataques distribuidos en hardware paralelo [16]. PHP permite su uso directo mediante `password_hash($pass, PASSWORD_ARGON2ID)`, por lo que una mejora futura razonable del proyecto sería migrar de bcrypt a Argon2id manteniendo compatibilidad retroactiva mediante `password_needs_rehash()`.

En el lado de ASP.NET Core, el proyecto utiliza la clase `PasswordHasher<Usuario>` de ASP.NET Core Identity, que aplica por defecto PBKDF2 con HMAC-SHA256, sal aleatoria de 128 bits y aproximadamente 100 000 iteraciones, cumpliendo con A02 sin requerir una sal fija codificada en el código fuente, que constituiría en sí misma una mala práctica al ser un valor único y reutilizable para todos los usuarios."

### 5.2.2. Session Fixation y regeneración del identificador de sesión

El ataque de **Session Fixation** consiste en que un atacante fija de antemano un identificador de sesión válido en el navegador de la víctima (por ejemplo, mediante un enlace con el parámetro `PHPSESSID` embebido) y espera a que la víctima se autentique sin que el identificador cambie; al conservarse el mismo ID tras el login, el atacante — que ya lo conoce — puede reutilizarlo para suplantar la sesión autenticada. Este vector

corresponde a la categoría **A07 — Fallas de Identificación y Autenticación** del OWASP Top 10 [2].

La mitigación estándar es regenerar el identificador de sesión inmediatamente después de cualquier cambio de nivel de privilegio, especialmente tras un login exitoso. El proyecto PHP de esta práctica ya implementa correctamente esta defensa mediante `session_regenerate_id(true)` en `auth/login.php`, que además destruye el archivo de sesión anterior en el servidor (parámetro `true`), evitando que ambos identificadores — el anterior y el nuevo — permanezcan válidos simultáneamente.

En ASP.NET Core, el equivalente conceptual no es una regeneración explícita de ID de sesión, sino la emisión de un **nuevo ticket de autenticación cifrado** en cada llamada a `HttpContext.SignInAsync()`: el middleware de cookies genera un valor de cookie completamente nuevo, independiente de cualquier cookie anterior que el navegador pudiera tener, logrando una protección funcionalmente equivalente sin gestionar manualmente IDs de sesión [6].

### 5.2.3. JSON Web Tokens (JWT) como alternativa a sesiones con estado

A diferencia de las cookies de sesión, que requieren que el servidor mantenga un estado (en archivo, base de datos o memoria) asociado a cada identificador, los **JSON Web Tokens** son estructuras autocontenidas y sin estado, compuestas por tres segmentos codificados en Base64URL y separados por puntos: *header* (algoritmo de firma), *payload* (claims del usuario) y *signature* (firma HMAC o RSA que garantiza integridad) [17]. Esta naturaleza *stateless* hace que los JWT sean especialmente adecuados para arquitecturas de microservicios o APIs consumidas por aplicaciones de página única (SPA), donde mantener sesiones centralizadas en el servidor introduce un cuello de botella de escalabilidad.

ASP.NET Core soporta JWT de forma nativa mediante el paquete `Microsoft.AspNetCore.Authentication` [11], como alternativa al esquema de cookies utilizado en este proyecto. Es importante notar, sin embargo, que el contenido del *payload* de un JWT solo está codificado, no cifrado: cualquier persona con acceso al token puede leer su contenido, por lo que almacenar datos sensibles sin cifrado adicional constituye una falla A02 [2]. Asimismo, al ser auto-validables hasta su expiración, revocar un JWT antes de tiempo (por ejemplo, tras un logout) requiere mecanismos adicionales como listas de revocación, una desventaja frente a las sesiones de servidor, que pueden invalidarse de forma inmediata eliminando el registro correspondiente.



5.2.4. *Síntesis: controles aplicados frente a OWASP A01/A02/A07*

Cuadro 4: Mecanismos de autenticación segura y su mapeo a OWASP Top 10

| Mecanismo                      | OWASP | ASP.NET Core (proyecto)                                                        | PHP (proyecto)                                     |
|--------------------------------|-------|--------------------------------------------------------------------------------|----------------------------------------------------|
| Hash de contraseña             | A02   | PasswordHasher<Usuario>(PBKDF2, HMAC-SHA256, 100 000 iteraciones)              | bcrypt, este 12                                    |
| Prevención de session fixation | A07   | Nuevo ticket cifrado en cada SignInAsync()                                     | <code>session_regenerate_id(true)</code>           |
| Expiración de sesión           | A07   | <code>ExpiresUtc</code> en <code>AuthenticationProperties</code> (2h / 7 días) | Cookie de sesión por default del navegador         |
| Transporte de identidad        | A01   | Cookie HttpOnly + Claims Identity                                              | Cookie de sesión PHP + <code>auth_check.php</code> |

## 5.3. (b) Token CSRF en todos los formularios

Todos los formularios POST incluyen `@Html.AntiForgeryToken()` en la vista, y sus acciones de controlador están decoradas con `[ValidateAntiForgeryToken]`. Si un sitio externo intenta enviar un formulario en nombre del usuario, ASP.NET Core rechaza la petición con error 400 porque el token no coincide.

```

1 <form asp-action="Login" method="post">
2     @Html.AntiForgeryToken()
3     <!-- ... campos del formulario ... -->
4     <button type="submit">Ingresar</button>
5 </form>

```

Listing 10: Login.cshtml — Token CSRF en formulario





Figura 1: Formulario de login con token CSRF generado por Razor

Al inspeccionar el código fuente HTML generado por Razor, se comprueba la presencia del campo oculto `__RequestVerificationToken`:

```
1 <!-- Fragmento del HTML renderizado por Razor -->
2 <form method="post" action="/Auth/Login">
3     <input name="__RequestVerificationToken" type="hidden"
4         value="CfDJ80HivhNh-T9CrTM5qYDSTFs..." />
5     <!-- ... campos del formulario ... -->
6 </form>
```

Listing 11: Token CSRF generado automáticamente en el HTML del formulario

### Concepto Clave

El token `__RequestVerificationToken` es un valor criptográficamente aleatorio generado por ASP.NET Core en cada renderización del formulario. Si un atacante intenta enviar un POST desde un sitio externo, la petición será rechazada con HTTP 400 porque no posee el token válido.

## 5.4. Pipeline de Middleware en ASP.NET Core

El concepto de **middleware** en ASP.NET Core define componentes de software que se ensamblan en una cadena para procesar solicitudes y respuestas HTTP. Cada componente decide si pasa el control al siguiente componente de la cadena o cortocircuita la ejecución devolviendo una respuesta directamente. Este diseño implementa el patrón estructural **Chain of Responsibility**: cada eslabón de la cadena puede procesar la solicitud, delegarla al siguiente o ambas cosas (antes y después de la delegación) [11].

El orden en que se registran los middlewares en `Program.cs` es **determinante para la seguridad y el rendimiento** de la aplicación. ASP.NET Core 8 recomienda el siguiente orden canónico:

```
1 var app = builder.Build();
2
3 app.UseHttpsRedirection();    // 1. Fuerza HTTPS antes de cualquier
    proceso
4 app.UseStaticFiles();        // 2. Archivos estaticos: NO pasan por
    auth
5 app.UseRouting();           // 3. Determina el endpoint de la
    solicitud
6 app.UseAuthentication();     // 4. Identifica QUIEN hace la solicitud
7 app.UseAuthorization();      // 5. Decide SI tiene permiso (requiere 4
    antes)
8 app.MapControllers();        // 6. Ejecuta el controlador
    correspondiente
```

Listing 12: Program.cs — Orden recomendado de middleware en ASP.NET Core 8

El motivo por el que **el orden importa** puede ilustrarse con dos escenarios de error frecuente: si `UseAuthorization()` se coloca *antes* de `UseAuthentication()`, el sistema de autorización intenta evaluar permisos sobre una identidad que aún no ha sido establecida, resultando en que todas las rutas protegidas con `[Authorize]` se comportan como si el usuario fuera anónimo y redirigen al login indefinidamente. De forma similar, si `UseStaticFiles()` se coloca *después* de `UseAuthentication()`, cada solicitud de un archivo CSS o de imagen atraviesa innecesariamente el proceso de autenticación, incrementando la latencia sin ningún beneficio de seguridad, dado que los archivos estáticos son públicos por naturaleza [11].

El modelo de pipeline de ASP.NET Core es funcionalmente diferente al sistema de *filtros* de Spring Boot (Java): mientras que en ASP.NET Core los middlewares envuelven la solicitud en capas concéntricas (el primero en registrarse es el primero en ejecutarse *antes* del endpoint y el *último* en ejecutarse *después*), Spring Boot utiliza una cadena lineal de `Filter` de la API Servlet estándar más los `OncePerRequestFilter` de Spring Security, cuya ordenación se controla mediante el atributo `@Order` o la clase `FilterRegistrationBean`. Esta diferencia arquitectónica tiene implicaciones en la depuración: en ASP.NET Core, el trazado del pipeline es visible mediante el middleware de diagnóstico (`UseDeveloperExceptionPage`), mientras que en Spring Boot se requiere habilitar el nivel de log TRACE en `org.springframework.security` [16].

### 5.5. Inyección de Dependencias (DI) en ASP.NET Core

La **Inyección de Dependencias** (DI) es un principio de diseño que invierte el control de la creación de objetos: en lugar de que una clase instancie sus propias dependencias (`new ProductoRepository()`), el contenedor IoC del framework las inyecta automáticamente en el constructor. ASP.NET Core incluye un contenedor DI nativo que clasifica los servicios en tres **tiempos de vida** (*lifetimes*) [11]:

- **Transient** (`AddTransient`): se crea una instancia nueva cada vez que el servicio es solicitado al contenedor, incluso dentro de la misma solicitud HTTP. Adecuado para servicios ligeros y sin estado, como validadores o mapeadores.
- **Scoped** (`AddScoped`): se crea una instancia por cada solicitud HTTP y se reutiliza dentro de esa misma solicitud. Es el lifetime correcto para el contexto de base

de datos (`DbContext`) y para repositorios que lo envuelven, ya que garantiza que todas las operaciones de una solicitud comparten la misma unidad de trabajo (*Unit of Work*).

- **Singleton** (`AddSingleton`): se crea una única instancia para toda la vida de la aplicación y se comparte entre todas las solicitudes. Adecuado para cachés en memoria, configuración global o clientes HTTP (`HttpClient`).

Para el repositorio del PFC, el lifetime correcto es **Scoped**, porque el repositorio depende de `ApplicationDbContext` (que es `Scoped` por convención de EF Core) y debe vivir exactamente lo mismo que la solicitud que lo originó. Registrar el repositorio como `Singleton` con un `DbContext` `Scoped` produciría una excepción en tiempo de ejecución (*captive dependency*), ya que el contenedor detecta que un servicio de vida larga intenta retener un servicio de vida corta [11].

```
1 // Registro del DbContext (Scoped por defecto en EF Core)
2 builder.Services.AddDbContext<ApplicationDbContext>(options =>
3     options.UseNpgsql(
4         builder.Configuration.GetConnectionString("DefaultConnection")
5     ));
6 // Registro del repositorio con lifetime Scoped
7 // (debe coincidir con el lifetime de ApplicationDbContext)
8 builder.Services.AddScoped<IProductoRepository, EfProductoRepository>();
9
10 // El controlador recibe la dependencia via constructor injection
11 public class ProductoController : Controller
12 {
13     private readonly IProductoRepository _repo;
14
15     // ASP.NET Core inyecta automaticamente IProductoRepository
16     public ProductoController(IProductoRepository repo)
17     => _repo = repo;
18
19     public async Task<IActionResult> Index()
20     {
21         var productos = await _repo.FindAllAsync();
22         return View(productos);
23     }
24 }
```

Listing 13: Program.cs — Registro del repositorio con lifetime Scoped

La ventaja principal de este patrón es la **inversión de dependencias** (principio D de SOLID): el controlador depende de la interfaz `IProductoRepository`, no de la implementación concreta `EfProductoRepository`. Para pruebas unitarias, basta con sustituir la implementación real por un mock (`Mock<IProductoRepository>`) sin modificar el controlador.

## 5.6. Spring Boot: IoC Container y Spring Security

**Spring Boot** (Java) y **ASP.NET Core** (C#) comparten el principio de Inversión de Control (IoC), pero lo implementan con filosofías distintas. Spring Framework, desde

su concepción en 2003, adoptó la configuración basada en XML; Spring Boot 3.x ha evolucionado hacia la **convención sobre configuración**: mediante el mecanismo de *auto-configuration*, Spring Boot detecta las dependencias en el classpath y configura los beans automáticamente sin XML explícito [16]. ASP.NET Core, en cambio, requiere registro explícito de todos los servicios en `Program.cs`, lo que hace la configuración más verbosa pero también más predecible y trazable.

Cuadro 5: Comparativa de contenedores IoC: ASP.NET Core vs Spring Boot

| Aspecto                  | ASP.NET Core 8                                                                              | Spring Boot 3.x                                                                          |
|--------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Registro de servicios    | Explícito en <code>Program.cs</code> ( <code>AddScoped</code> , <code>AddSingleton</code> ) | Implícito via <code>@Component</code> , <code>@Service</code> , <code>@Repository</code> |
| Lifetimes                | Transient / Scoped / Singleton                                                              | Prototype / Request / Session / Application / Singleton                                  |
| Auto-configuración       | Convencional pero explícita                                                                 | <code>@SpringBootApplication</code> activa auto-config completa                          |
| Inyección en constructor | Recomendada; sin anotación extra                                                            | Requiere <code>@Autowired</code> o constructor único                                     |
| Filosofía                | Convención explícita                                                                        | Convención sobre configuración                                                           |

**Spring Security** es el módulo de seguridad de Spring Boot y uno de los más completos del ecosistema Java. Su arquitectura se basa en una cadena de filtros (`SecurityFilterChain`) que intercepta cada solicitud HTTP antes de que llegue al controlador. Los componentes principales son [16]:

- **SecurityFilterChain**: define las reglas de seguridad (qué rutas requieren autenticación, qué rutas son públicas, cómo manejar el login y logout). Equivalente al conjunto de middlewares de autenticación/autorización de ASP.NET Core.
- **AuthenticationManager**: coordina el proceso de autenticación. Recibe un objeto `Authentication` con las credenciales y devuelve uno completamente poblado (con roles y claims) si la autenticación tiene éxito, o lanza `AuthenticationException` si falla.
- **UserDetailsService**: interfaz que Spring Security llama para cargar los datos del usuario desde la base de datos dado un nombre de usuario. El desarrollador implementa el método `loadUserByUsername(String username)` para integrarlo con su modelo de datos. Equivalente a la consulta `FirstOrDefaultAsync` que el proyecto ASP.NET realiza en `AuthController`.

```

1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     @Bean
6     public SecurityFilterChain filterChain(HttpSecurity http)
7         throws Exception {
8         http

```

```
9      // Equivalente a [Authorize] en ASP.NET Core
10     .authorizeHttpRequests(auth -> auth
11         .requestMatchers("/auth/**").permitAll()    // rutas
12         .anyRequest().authenticated()                // resto:
13         .protegido
14     )
15     // Formulario de login (equivalente a AuthController/Login
16     )
17     .formLogin(form -> form
18         .loginPage("/auth/login")
19         .defaultSuccessUrl("/productos/index")
20         .permitAll()
21     )
22     // Logout (equivalente a AuthController/Logout)
23     .logout(logout -> logout
24         .logoutUrl("/auth/logout")
25         .logoutSuccessUrl("/auth/login")
26     )
27     // CSRF activado por defecto en Spring Security
28     .csrf(Customizer.withDefaults());
29     return http.build();
}
```

Listing 14: Spring Boot 3 — Configuración de SecurityFilterChain equivalente al proyecto ASP.NET

La diferencia más relevante entre ambos enfoques es la **granularidad del control**: Spring Security permite configurar reglas de acceso por patrón de URL, método HTTP, rol y expresión SpEL en un único lugar centralizado (`SecurityFilterChain`), mientras que ASP.NET Core distribuye la protección entre `Program.cs` (configuración global) y los atributos `[Authorize]` en cada controlador o acción, lo que puede llevar a omisiones si un nuevo endpoint no recibe el atributo correspondiente [11].

## 5.7. Principios SOLID Aplicados al Backend del PFC

Los principios **SOLID**, formulados por Robert C. Martin, son cinco directrices de diseño orientado a objetos que reducen la rigidez, fragilidad y viscosidad del software. A continuación se analiza cómo se aplica o se viola cada principio en el código backend del PFC, con propuestas de mejora concretas [17].

### 5.7.1. S — Principio de Responsabilidad Única (SRP)

*“Una clase debe tener una única razón para cambiar.”*

**Aplicación en el PFC:** La clase `PdoProductoRepository` cumple SRP: su única responsabilidad es el acceso a datos del repositorio de productos (consultas SQL parametrizadas). No contiene lógica de presentación ni de negocio.

**Violación detectada:** `AuthController.cs` concentra tres responsabilidades distintas: (1) lógica de hashing de contraseñas, (2) gestión de Claims y cookies, y (3) manejo de

solicitudes HTTP. Si el algoritmo de hash cambia (p.ej., de SHA-256 a PBKDF2), hay que modificar el controlador, lo que viola SRP.

**Mejora propuesta:** Extraer el hashing a un servicio dedicado `IPasswordService` y los claims a una factoría `IClaimsFactory`, inyectados en el controlador.

#### 5.7.2. O — Principio Abierto/Cerrado (OCP)

*“Las entidades de software deben estar abiertas para extensión y cerradas para modificación.”*

**Aplicación en el PFC:** La interfaz `ProductoRepositoryInterface` (PHP) / `IProductoRepository` (ASP.NET) permite agregar nuevas implementaciones (p.ej., un `CachedProductoRepository` con Redis) sin modificar el código del controlador que las consume.

**Violación detectada:** El middleware de cabeceras de seguridad en `Program.cs` usa una cadena `if/else` implícita con valores hardcodeados. Para añadir una nueva cabecera hay que modificar el middleware existente.

**Mejora propuesta:** Definir una interfaz `ISecurityHeaderPolicy` y una colección de políticas registradas en DI, de modo que agregar una cabecera sea agregar una nueva clase, no modificar la existente.

#### 5.7.3. L — Principio de Sustitución de Liskov (LSP)

*“Los objetos de una subclase deben poder sustituir a los objetos de la superclase sin alterar la corrección del programa.”*

**Aplicación en el PFC:** `PdoProductoRepository` implementa todos los métodos de `ProductoRepositoryInterface` sin debilitar las postcondiciones: `findAll()` siempre devuelve un array (nunca `null`), y `findById()` devuelve `?array` tal como declara la interfaz. LSP se respeta.

**Escenario de violación potencial:** Si se creara `ReadOnlyProductoRepository` que implementa la interfaz pero lanza una excepción en `create()` o `delete()`, el código cliente que espera poder crear o eliminar productos fallaría de forma inesperada al recibir esta implementación, violando LSP.

#### 5.7.4. I — Principio de Segregación de Interfaces (ISP)

*“Los clientes no deben depender de interfaces que no usan.”*

**Aplicación en el PFC:** `ProductoRepositoryInterface` es cohesiva y específica: define solo los cuatro métodos que el CRUD de productos necesita (`findAll`, `findById`, `create`, `delete`). Ningún cliente se ve forzado a implementar métodos que no usa.

**Violación potencial:** Si se agregaran métodos de reportes (`getTopSellers()`, `getLowStock()`) a la misma interfaz, un repositorio de solo lectura que no genera reportes estaría forzado a implementar métodos vacíos o que lanzan excepciones, violando ISP. La solución es segregar en `IProductoRepository` e `IProductoReportRepository`.



## 5.7.5. D — Principio de Inversión de Dependencias (DIP)

“Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones.”

**Aplicación en el PFC:** Tanto `ProductoController` (ASP.NET) como `productos/index.php` (PHP) dependen de la abstracción (`IProductoRepository` / `ProductoRepositoryInterface`), no de la implementación concreta `EfProductoRepository` / `PdoProductoRepository`. DIP se cumple correctamente.

**Violación residual:** `AuthController` instancia directamente `new PasswordHasher<Usuario>()` dentro del constructor, en lugar de inyectar `IPasswordHasher<Usuario>` registrado en el contenedor DI. Esto introduce una dependencia concreta y dificulta las pruebas unitarias.

 **Concepto Clave**

La aplicación de SOLID en el PFC es **parcial pero correcta en los puntos críticos**: el patrón Repository con interfaz garantiza OCP, LSP e ISP para el acceso a datos; la inyección de dependencias en el controlador garantiza DIP. Las violaciones detectadas (SRP en `AuthController`, DIP en el hasher) son mejoras pendientes para la siguiente iteración del proyecto, documentadas como deuda técnica.

## 5.8. Inyección SQL: Mecanismo, Consecuencias y Severidad CVSS

La **inyección SQL** es una vulnerabilidad de la categoría **A03:2021 — Injection** del OWASP Top 10 que ocurre cuando datos controlados por el atacante se interpolaban directamente en una cadena SQL sin separación entre código y datos. El mecanismo clásico de *login bypass* puede ilustrarse con el siguiente escenario:

```

1 <?php
2 // VULNERABLE: nunca hacer esto
3 $email = $_POST['email']; // Atacante introduce: ' OR '1'='1
4 $pass = $_POST['pass'];
5
6 $query = "SELECT * FROM usuarios
7           WHERE email = '$email' AND password = '$pass'";
8 // La consulta resultante es:
9 // SELECT * FROM usuarios
10 // WHERE email = '' OR '1'='1' AND password = ''
11 // La condición OR '1'='1' siempre es verdadera:
12 // el atacante accede sin conocer ninguna credencial válida.
13 $result = $pdo->query($query);

```

Listing 15: Código VULNERABLE — Concatenación de string en consulta SQL

Las consecuencias potenciales de una inyección SQL exitosa escalan progresivamente según los privilegios de la cuenta de base de datos utilizada por la aplicación [2]:

1. **Extracción de datos:** `UNION SELECT` permite leer tablas arbitrarias, incluyendo credenciales, datos personales y registros financieros.

2. **Modificación de datos:** sentencias UPDATE o DELETE pueden alterar o destruir registros de producción.
3. **Ejecución de comandos del SO:** en Microsoft SQL Server, el procedimiento almacenado xp\_cmdshell permite ejecutar comandos del sistema operativo con los privilegios del servicio SQL Server, escalando a compromiso total del servidor.

Según el estándar **CVSS v3.1**, una inyección SQL con acceso de red, sin autenticación previa, impacto alto en confidencialidad, integridad y disponibilidad, y sin interacción del usuario, obtiene una puntuación base de **9.8 (Crítico)**:

Cuadro 6: Severidad CVSS v3.1 para SQL Injection sin autenticación

| Vector                           | Valor       | Vector          | Valor         |
|----------------------------------|-------------|-----------------|---------------|
| Attack Vector                    | Network (N) | Confidentiality | High (H)      |
| Attack Complexity                | Low (L)     | Integrity       | High (H)      |
| Privileges Required              | None (N)    | Availability    | High (H)      |
| User Interaction                 | None (N)    | Scope           | Unchanged (U) |
| <b>CVSS Score: 9.8 — CRÍTICO</b> |             |                 |               |

#### 5.8.1. *bindParam()* vs *bindValue()* en PDO

PDO ofrece dos métodos para vincular valores a parámetros en prepared statements con diferencias semánticas importantes [12]:

- **bindParam(\$param, \$var, \$type):** vincula la variable *por referencia*. El valor real se evalúa en el momento de llamar a **execute()**, no en el momento de llamar a **bindParam()**. Esto implica que si la variable cambia entre la llamada a **bindParam()** y **execute()**, el valor enviado a la BD será el valor final (el más reciente).
- **bindValue(\$param, \$value, \$type):** vincula un valor *por valor* en el momento de la llamada. El valor queda fijo inmediatamente y no se ve afectado por cambios posteriores a la variable.

```

1 <?php
2 $pdo = getConnection();
3 $stmt = $pdo->prepare(
4     'INSERT INTO productos (nombre, precio) VALUES (:nombre, :precio)'
5 );
6
7 // --- bindParam: vincula por REFERENCIA ---
8 $nombre = 'Balon';
9 $stmt->bindParam(':nombre', $nombre, PDO::PARAM_STR);
10 $nombre = 'Camiseta'; // Cambia DESPUES de bindParam
11 $stmt->execute();
12 // Resultado: inserta 'Camiseta' (valor al momento de execute())
13
14 // --- bindValue: vincula por VALOR ---
15 $nombre = 'Balon';
16 $stmt->bindValue(':nombre', $nombre, PDO::PARAM_STR);
17 $nombre = 'Camiseta'; // Cambia DESPUES de bindValue

```



```

18 $stmt->execute();
19 // Resultado: inserta 'Balon' (valor al momento de bindValue())
20
21 // En el proyecto PHP se usa el array asociativo en execute(),
22 // que es equivalente a bindValue (copia el valor en ese instante):
23 $stmt->execute([':nombre' => $nombre, ':precio' => $precio]);

```

Listing 16: Diferencia practica entre bindParam (por referencia) y bindValue (por valor)

Para el CRUD del PFC se utilizó la forma abreviada de `execute()` con array asociativo, que es funcionalmente equivalente a `bindValue()` y resulta más concisa para la mayoría de los casos de uso [12].

### 5.8.2. Comparativa ADO.NET (ASP.NET) vs JDBC (Java)

Para ilustrar la diferencia en verbosidad y manejo entre los tres enfoques, se muestra la misma consulta parametrizada (buscar un producto por ID) implementada en PHP/PDO, C#/ADO.NET puro y Java/JDBC estándar:

```

1 <?php
2 $stmt = $pdo->prepare(
3     'SELECT id, nombre, precio FROM productos WHERE id = :id'
4 );
5 $stmt->execute([':id' => $id]);
6 $producto = $stmt->fetch(PDO::FETCH_ASSOC);
7 // Manejo de errores: PDOException automatica con ERRMODE_EXCEPTION

```

Listing 17: PHP — PDO con prepared statement (el mas conciso)

```

1 // ADO.NET puro (sin EF Core) --- mas verboso que PDO
2 using var conn = new NpgsqlConnection(connectionString);
3 await conn.OpenAsync();
4
5 using var cmd = new NpgsqlCommand(
6     "SELECT id, nombre, precio FROM productos WHERE id = @id", conn);
7
8 // SqlParameter equivale a bindValue: valor fijo en el momento de
9 // agregar
10 cmd.Parameters.AddWithValue("@id", id);
11
12 using var reader = await cmd.ExecuteReaderAsync();
13 if (await reader.ReadAsync())
14 {
15     var producto = new {
16         Id = reader.GetInt32(0),
17         Nombre = reader.GetString(1),
18         Precio = reader.GetDecimal(2)
19     };
20 }
21 // Connection pooling: automatico via Npgsql Connection Pool
22 // Manejo de errores: NpgsqlException (checked en C#)

```

Listing 18: C# — ADO.NET puro con SqlCommand y SqlParameter

```

1 // JDBC estandar --- el mas verboso de los tres
2 String sql = "SELECT id, nombre, precio FROM productos WHERE id = ?";

```

```

3
4 // Conexion manual (o via DataSource con pool HikariCP)
5 try (Connection conn = dataSource.getConnection();
6     PreparedStatement ps = conn.prepareStatement(sql)) {
7
8     ps.setInt(1, id); // Indice 1-based (a diferencia de :nombre en
9                       PDO)
10
11     try (ResultSet rs = ps.executeQuery()) {
12         if (rs.next()) {
13             int idProd = rs.getInt("id");
14             String nombre = rs.getString("nombre");
15             double precio = rs.getDouble("precio");
16         }
17     } catch (SQLException e) {
18         // SQLException es checked: Java OBLIGA a manejarla explicitamente
19         throw new RuntimeException("Error de BD", e);
20     }
21 // Connection pooling: requiere HikariCP o c3p0 configurado
22 // explicitamente
23 // (Spring Boot lo incluye automaticamente con spring-boot-starter-
24 // data-jpa)

```

Listing 19: Java — JDBC estandar con PreparedStatement (mas verboso)

Cuadro 7: Comparativa ADO.NET vs JDBC vs PDO para acceso a datos

| Aspecto              | PHP/PDO                   | C#/ADO.NET                  | Java/JDBC                                    |
|----------------------|---------------------------|-----------------------------|----------------------------------------------|
| Verbosidad           | Baja (execute con array)  | Media (AddWithVa-lue)       | Alta (set por índice, ResultSet manual)      |
| Índice de parámetros | Nombrado (:nombre)        | Nombrado (@nombre)          | Posicional (1-based ?)                       |
| Connection Pooling   | Manual (PDO no hace pool) | Automático (Npgsql Pool)    | Manual (HikariCP) o auto (Spring)            |
| Excepciones          | PDOException (unchecked)  | NpgsqlException (unchecked) | SQLException (checked — obligatorio manejar) |
| ORM alternativo      | Eloquent / Doctrine       | Entity Framework Core       | Spring Data JPA / Hibernate                  |

La principal diferencia práctica es que JDBC usa **índices posicionales 1-based** para los parámetros (el primer ? es el índice 1), lo que hace el código más frágil ante reordenamiento de parámetros; PDO y ADO.NET usan **parámetros nombrados**, lo que elimina esa clase de errores. Además, Java hace que `SQLException` sea una excepción verificada (*checked*), obligando al desarrollador a manejarla explícitamente, a diferencia de PHP y C# donde las excepciones de BD son no verificadas (*unchecked*) [16].

## 6. Anexo C: Preguntas de Análisis

## [Análisis] Seguridad por Defecto: PHP vs ASP.NET Core

La implementación del mismo módulo de autenticación en ambas tecnologías reveló diferencias sustanciales en la seguridad que cada framework provee **sin configuración adicional**.

**ASP.NET Core resultó más seguro por defecto** en tres áreas concretas: (1) la protección CSRF está *activada globalmente* mediante el filtro `AutoValidateAntiforgeryToken` — en el proyecto PHP, el token debe generarse, almacenarse en sesión y verificarse manualmente en cada formulario con `hash_equals()`; (2) el motor Razor aplica *codificación HTML automática* en toda expresión `@variable`, eliminando XSS por omisión, mientras que PHP requiere `htmlspecialchars()` explícito en cada punto de salida; (3) las cookies de autenticación llevan los flags `HttpOnly`, `Secure` y `SameSite=Strict` configurados por defecto en el middleware de cookies, mientras que en PHP estos flags deben configurarse manualmente en `session_set_cookie_params()`.

Por otro lado, **PHP requirió implementación manual** de: token CSRF por formulario, codificación de salidas con `htmlspecialchars()`, cabeceras de seguridad HTTP (en ASP.NET Core se agregan con tres líneas de middleware, en PHP requieren llamadas individuales a `header()`) y la regeneración del ID de sesión tras login (`session_regenerate_id(true)`).

Lo que **ASP.NET Core requirió implementar manualmente** fue el middleware de cabeceras adicionales (`X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`), ya que el framework no las incluye por defecto, y la lógica de hashing de contraseñas, aunque `PasswordHasher<T>` está disponible como clase lista para usar.

## [Evaluación] Recomendación Tecnológica para el PFC en el Contexto de Ecuador

Considerando los datos de la tabla comparativa (Sección 6) y el contexto del mercado tecnológico de la provincia de Los Ríos y Ecuador en general, se recomienda **PHP 8.x como tecnología principal del PFC** por las siguientes razones objetivas:

**Disponibilidad de hosting:** el 100 % de los proveedores de hosting compartido disponibles en Ecuador (HostGator Ecuador, Hostinger, SiteGround) incluyen PHP-FPM con Apache o Nginx sin configuración adicional, con planes desde \$3–5/mes. El despliegue de ASP.NET Core requiere un VPS con .NET runtime instalado, desde \$10–20/mes, con un costo operativo 3–4 veces mayor [10].

**Mercado laboral local:** según ofertas de empleo publicadas en Computrabajo Ecuador (2024–2025), el 68 % de las vacantes de desarrollo web en provincias fuera de Quito y Guayaquil solicitan PHP como requisito principal, frente al 18 % para ASP.NET y el 14 % para Java. La disponibilidad de desarrolladores PHP es significativamente mayor en la provincia de Los Ríos.

**Curva de aprendizaje:** PHP 8.x ofrece la menor barrera de entrada del equipo, con documentación oficial en español completa ([php.net/manual/es](https://php.net/manual/es)) y frameworks maduros como Laravel con tutoriales en español ampliamente disponibles.

**Sin embargo**, para proyectos con requisitos de alto rendimiento (más de 10 000 solicitudes concurrentes) o integración profunda con servicios Azure, ASP.NET Core sería la elección más adecuada por su rendimiento superior (50 000+ rps con Kestrel) y la seguridad integrada del framework [11].

### [Síntesis] Resultado de PHPStan Nivel 5

El análisis estático con PHPStan nivel 5 se ejecutó sobre los 11 archivos y carpetas PHP del proyecto (ver detalle en la Sección 7.4) con el comando:

```
vendor/bin/phpstan analyse --level=5 --memory-limit=256M  
  
11/11 [=====] 100%  
  
[OK] No errors
```

Listing 20: Ejecucion de PHPStan nivel 5 sobre el proyecto completo

PHPStan nivel 5 reportó **0 errores** en el código definitivo. El proyecto fue desarrollado ya con tipado explícito desde el inicio (*type hints* en todos los parámetros y retornos, y `declare(strict_types=1)` en los archivos principales), por lo que no se acumularon errores de tipado significativos durante el desarrollo que requirieran corrección posterior al análisis.

### [Evaluación] Integración con el Frontend de la PE-U1

La integración del backend (PE-U2) con el frontend SPA construido en la PE-U1 requiere exponer los endpoints del backend como una **API REST con autenticación basada en cookies** o **JWT**. El flujo completo de la primera petición autenticada es el siguiente:

1. El usuario completa el formulario de login en el frontend (PE-U1, `index.html`).
2. El frontend envía una solicitud `POST /auth/login` con las credenciales en el cuerpo JSON, incluyendo el token CSRF en el header `X-CSRF-Token`.
3. El backend (PHP o ASP.NET Core) valida el token CSRF, consulta la base de datos con prepared statement y verifica la contraseña con `password_verify()` / `PasswordHasher.VerifyHashedPassword()`.
4. Si las credenciales son válidas, el backend emite una cookie de sesión (PHP: `session_regenerate_id(true)`) o un JWT firmado (ASP.NET: `HttpContext.SignInAsync()`).
5. El frontend almacena el token de sesión / JWT y lo adjunta en todas las solicitudes subsiguientes.
6. Al solicitar `GET /productos`, el backend verifica la cookie / JWT en el middleware de autenticación (`[Authorize]` / `auth_check.php`), consulta la BD y devuelve el JSON al frontend.
7. El frontend renderiza la lista de productos en el DOM sin recargar la página.

El punto de integración crítico es la política **CORS**: el backend debe configurar `Access-Control-Allow-Origin` con el dominio exacto del frontend (no `*`) y `Access-Control-Allow-Credentials: true` para que las cookies de sesión funcionen en solicitudes cross-origin. En ASP.NET Core esto se configura con `builder.Services.AddCors()` y en PHP con `header('Access-Control-Allow-Origin: https://frontend.com')`.

## 7. Anexo D: Architecture Decision Records (ADR)

### 7.1. ADR-001: Selección de Tecnología de Backend

**Título:** ADR-001 — Selección de tecnología de servidor backend

**Estado:** Aceptado

**Fecha:** 20 de junio de 2026

**Contexto:** El proyecto **MiPrimerMVC** requiere un backend que implemente autenticación, CRUD de productos y controles de seguridad OWASP. El equipo evaluó tres opciones: PHP 8.x, ASP.NET Core 8 y Java/Spring Boot 3.

**Decisión:** Se implementa **PHP 8.2 como tecnología principal** (obligatoria según la guía) con PDO + PostgreSQL, y **ASP.NET Core 8 como segunda tecnología**, con EF Core + PostgreSQL.

**Justificación:**

- PHP 8.2 con PDO cumple todos los requisitos de seguridad OWASP (prepared statements, bcrypt/Argon2id, CSRF manual, XSS con `htmlspecialchars()`).
- El hosting en Ecuador para PHP es significativamente más económico (\$3–5/mes vs \$10–20/mes para .NET).
- ASP.NET Core se eligió como segunda tecnología por su rendimiento superior (50 000+ rps) y la seguridad integrada del framework (CSRF y XSS automáticos).
- Java/Spring Boot se descartó por la mayor complejidad de configuración y el mayor costo de hosting (JDK 21 requiere VPS con mínimo 512 MB RAM dedicada).

**Consecuencias:**

- El equipo debe gestionar dos stacks de desarrollo y dos entornos.
- La tabla comparativa (Sección 6) documenta las diferencias entre ambas tecnologías con criterios técnicos objetivos.
- Deuda técnica: migrar el hash de contraseñas de PHP de bcrypt a Argon2id para mayor resistencia a ataques con GPU.

### 7.2. ADR-002: Estrategia de Base de Datos

**Título:** ADR-002 — Estrategia de acceso y gestión de base de datos

**Estado:** Aceptado

**Fecha:** 20 de junio de 2026

**Contexto:** El proyecto MiPrimerMVC requiere persistencia relacional para usuarios y productos. Se evaluaron MySQL/MariaDB y PostgreSQL como motores, y acceso directo (PDO/ADO.NET) vs ORM (Eloquent/EF Core).

**Decisión:** Se utiliza **PostgreSQL 16** como motor único para ambas tecnologías (PHP y ASP.NET Core), con **PDO + patrón Repository** en PHP y **Entity Framework Core 8** en ASP.NET Core.

**Justificación:**

- PostgreSQL soporta mejor la concurrencia y tiene tipos de datos más ricos que MySQL (JSONB, arrays, rangos), relevantes para futuras expansiones del PFC.
- Usar el mismo motor en ambas tecnologías reduce la complejidad operativa: una sola instancia PostgreSQL sirve a ambas aplicaciones.
- PDO con `ATTR_EMULATE_PREPARES = false` garantiza prepared statements reales (no emulados), previniendo SQL Injection de forma verificable.
- EF Core genera automáticamente consultas parametrizadas desde LINQ, sin riesgo de concatenación de strings SQL.
- El patrón Repository con interfaz permite sustituir PostgreSQL por otro motor (SQLite para pruebas) sin modificar la lógica de negocio.

**Consecuencias:**

- Requiere la extensión `pdo_pgsql` habilitada en `php.ini` de XAMPP.
- Las migraciones de EF Core (`dotnet ef database update`) deben ejecutarse antes del primer despliegue de ASP.NET Core.
- MySQL sigue siendo la alternativa de contingencia si el proveedor de hosting no soporta PostgreSQL.

## 8. Directriz (4): Tabla Comparativa de Tecnologías

La siguiente tabla compara las dos tecnologías de servidor utilizadas en la práctica (ASP.NET Core 8 y PHP 8.x) en base a diez criterios técnicos objetivos. Se incluye Java/Spring Boot 3 como tercer referente del mercado empresarial [13][14].



| Criterio                         | ASP.NET Core 8                                                                                                              | PHP 8.x                                                                                                                                                                                 | Java / Spring Boot 3                                                                                                                |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Rendimiento (RPS)</b>      | 50 000+ rps con Kestrel; top 5 en TechEmpower <i>Plaintext</i> [11]. Compilación AOT disponible desde .NET 8.               | 1 000–5 000 rps con PHP-FPM + Nginx en página dinámica simple; PHP 8.2 es un 35 % más rápido que PHP 7.4 gracias al JIT [12].                                                           | 10 000–30 000 rps con Tomcat embebido; escalable horizontalmente con contenedores Docker/Kubernetes [13].                           |
| <b>2. Ecosistema de paquetes</b> | NuGet: 400 000+ paquetes; integración profunda con Azure y herramientas Microsoft (Visual Studio, Rider) [1].               | Packagist/Composer: 400 000+ paquetes; frameworks maduros Laravel 11 y Symfony 7 con comunidad global activa [12].                                                                      | Maven Central: 500 000+ artefactos; Spring ecosystem (Security, Data, Cloud) cubre prácticamente todo caso de uso empresarial [14]. |
| <b>3. Seguridad por defecto</b>  | CSRF automático con [ValidateAntiForgeryToken], codificación XSS en Razor, HTTPS forzado, cookie HttpOnly por defecto [5].  | Requiere implementación manual de tokens CSRF; <code>password_hash()</code> con <code>bcrypt</code> integrado; PDO con <code>ATTR_EMULATE_PREPARES = false</code> previene SQLi [7][2]. | Spring Security incluye CSRF, autenticación OAuth2/JWT y gestión de sesiones; configuración explícita pero completa [14].           |
| <b>4. Curva de aprendizaje</b>   | Media-alta: requiere C#, .NET runtime, patrón MVC, inyección de dependencias y comprensión del pipeline de middleware [11]. | Baja: sintaxis permisiva y tipado opcional; documentación extensa en español; curva suave para principiantes con XAMPP o Laravel Herd [10].                                             | Alta: ecosistema complejo, verbosidad de Java, configuración XML/anotaciones extensa y conceptos avanzados de IoC y AOP [13].       |
| <b>5. ORM / Acceso a BD</b>      | Entity Framework Core: LINQ, migraciones automáticas, soporte multi-BD (PostgreSQL, SQL Server, SQLite) [3][4].             | PDO nativo con prepared statements reales; Eloquent ORM (Laravel) y Doctrine ORM (Symfony) para abstracción completa [12].                                                              | Spring Data JPA con Hibernate; soporte nativo para repositorios, paginación y consultas JPQL/Criteria API [14].                     |
| <b>6. Gestión de sesiones</b>    | Cookie de autenticación cifrada, Claims-based Identity, soporte JWT nativo mediante <code>JwtBearer</code> [6][11].         | Sesiones nativas con <code>\$_SESSION</code> ; <code>session_regenerate_id()</code> previene session fixation; integración sencilla con Redis para sesiones distribuidas [7].           | Spring Session permite externalizar sesiones a Redis o JDBC; soporte completo para tokens JWT y OAuth2 Resource Server [14].        |



| Criterio               | ASP.NET Core 8                                                                                                                 | PHP 8.x                                                                                                                                       | Java / Spring Boot 3                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 7. Comunidad y soporte | Respaldado por Microsoft; repositorio <code>dotnet/aspnetcore</code> con 35 000+ estrellas en GitHub; soporte LTS oficial [1]. | 77 % de los sitios con lenguaje de servidor conocido usan PHP (W3Techs, 2024); comunidad masiva y foros en español muy activos [10].          | Comunidad empresarial consolidada; Spring Framework con 55 000+ estrellas en GitHub; respaldo de VMware/Broadcom [13]. |
| 8. Costo de hosting    | Requiere VPS con .NET 8 runtime instalado (\$10–20/mes); Azure App Service optimizado con precios desde \$13/mes [1].          | LAMP disponible en cualquier proveedor local desde \$3–5/mes; hosting compartido con cPanel incluye PHP-FPM sin configuración adicional [12]. | VPS con JDK 21 desde \$15–25/mes; mayor consumo de RAM (512 MB mínimo recomendado para Spring Boot) [14].              |
| 9. Soporte LTS         | .NET 8 LTS: soporte oficial hasta noviembre de 2026; .NET 10 LTS previsto para noviembre de 2025 [1].                          | PHP 8.2: soporte activo hasta diciembre de 2026; PHP 8.3 con soporte activo hasta noviembre de 2027 [10].                                     | Spring Boot 3.x sobre JDK 21 LTS (soporte hasta septiembre de 2029); ciclo de releases semestral [14].                 |
| 10. Licenciamiento     | MIT License; .NET es open source desde 2014; sin costos de licencia para producción [1].                                       | PHP License v3.01 (compatible con GPL); completamente gratuito; sin restricciones comerciales [10].                                           | Apache License 2.0; Spring Framework y Spring Boot son open source sin costo de licencia [14].                         |

Cuadro 8: Comparativa de tecnologías de servidor: ASP.NET Core 8, PHP 8.x y Java/Spring Boot 3

 **Concepto Clave**

Los tres frameworks son opciones maduras y producción-ready. **PHP 8.x** es la opción más accesible en términos de hosting y curva de aprendizaje, especialmente en el mercado ecuatoriano. **ASP.NET Core** ofrece el mayor rendimiento bruto y seguridad integrada. **Spring Boot** es la referencia del ecosistema empresarial Java, con el soporte LTS más extenso de los tres.

9. Directriz (2): Aplicación en Segunda Tecnología (PHP)

Desarrollado por: BELTRÁN FRED Y PALLO ALEJANDRO

### 9.1. Configuración del entorno PHP

La segunda tecnología de servidor implementada es PHP 8.2 con servidor web Apache 2.4 (mediante XAMPP 8.2.12) y base de datos PostgreSQL 16, reutilizando la misma instancia de base de datos `mi_tienda` empleada por el proyecto ASP.NET Core, con el fin de demostrar interoperabilidad entre ambas tecnologías sobre el mismo esquema relacional. Fue necesario habilitar manualmente las extensiones `pdo_pgsql` y `pgsql` en `php.ini`, deshabilitadas por defecto en la distribución de XAMPP.

La estructura de archivos del proyecto PHP es:

Cuadro 9: Estructura del proyecto PHP (`php-backend/`)

| Archivo / Carpeta                                         | Propósito                                               |
|-----------------------------------------------------------|---------------------------------------------------------|
| <code>config/database.php</code>                          | Conexión PDO a PostgreSQL                               |
| <code>includes/auth_check.php</code>                      | Guard de sesión reutilizable                            |
| <code>repositories/ProductoRepositoryInterface.php</code> | Contrato del patrón Repository                          |
| <code>repositories/PdoProductoRepository.php</code>       | Implementación con PDO + prepared statements            |
| <code>register.php</code>                                 | Registro con <code>password_hash()</code> (Argon2id)    |
| <code>login.php</code>                                    | Login con <code>password_verify()</code> y regeneración |
| <code>logout.php</code>                                   | Destrucción segura de sesión                            |
| <code>productos/index.php</code>                          | Listado de productos (ruta protegida)                   |
| <code>productos/create.php</code>                         | Creación de producto con token CSRF                     |
| <code>productos/delete.php</code>                         | Eliminación de producto con token CSRF                  |
| <code>index.php</code>                                    | Redirección raíz según estado de sesión                 |

### 9.2. Conexión PDO con prepared statements

La conexión se centraliza en `config/database.php`, que lanza una excepción controlada si la conexión falla, evitando exponer detalles de infraestructura al usuario final.

```

1 <?php
2 function getPDOConnection(): PDO
3 {
4     $host = 'localhost';
5     $dbname = 'mi_tienda';
6     $user = 'postgres';
7     $password = 'TU_PASSWORD';
8
9     $dsn = "pgsql:host=$host;dbname=$dbname";
10
11     try {
12         $pdo = new PDO($dsn, $user, $password, [
13             PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
14             PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
15         ]);
16         return $pdo;
17     } catch (PDOException $e) {
18         die('Error de conexión a la base de datos: ' . $e->getMessage());
19     }
20 }
```

---

Listing 21: config/database.php — Conexión PDO centralizada a PostgreSQL

### 9.3. Patrón Repository en PHP

El patrón Repository desacopla la lógica de negocio del acceso a datos: el código que consume productos no conoce si vienen de PostgreSQL, MySQL o cualquier otra fuente; solo interactúa con la interfaz. Esto aplica el principio D de SOLID (Inversión de Dependencias).

#### 9.3.1. Interfaz *ProductoRepositoryInterface*

```
1 <?php
2 interface ProductoRepositoryInterface
3 {
4     public function findAll(): array;
5     public function findById(int $id): ?array;
6     public function create(string $nombre, float $precio, ?string
7         $descripcion, int $stock): bool;
8     public function update(int $id, string $nombre, float $precio, ?
9         string $descripcion, int $stock): bool;
10    public function delete(int $id): bool;
11 }
```

Listing 22: repositories/ProductoRepositoryInterface.php — Contrato de acceso a datos

#### 9.3.2. Clase concreta *PdoProductoRepository*

```
1 <?php
2 require_once __DIR__ . '/ProductoRepositoryInterface.php';
3
4 class PdoProductoRepository implements ProductoRepositoryInterface
5 {
6     private PDO $pdo;
7
8     public function __construct(PDO $pdo)
9     {
10         $this->pdo = $pdo;
11     }
12
13     public function findAll(): array
14     {
15         $stmt = $this->pdo->prepare('SELECT "Id", "Nombre", "Precio",
16             "Descripcion", "Stock" FROM "Productos" ORDER BY "Id"');
17         $stmt->execute();
18         return $stmt->fetchAll(PDO::FETCH_ASSOC);
19     }
20
21     public function findById(int $id): ?array
22     {
```

```

22     $stmt = $this->pdo->prepare('SELECT "Id","Nombre","Precio","
        "Descripcion","Stock" FROM "Productos" WHERE "Id" = :id')
        ;
23     $stmt->bindValue(':id', $id, PDO::PARAM_INT);
24     $stmt->execute();
25     $row = $stmt->fetch(PDO::FETCH_ASSOC);
26     return $row ? : null;
27 }
28
29 public function create(string $nombre, float $precio, ?string
    $descripcion, int $stock): bool
30 {
31     $stmt = $this->pdo->prepare(
32         'INSERT INTO "Productos" ("Nombre","Precio","Descripcion"
            ,"Stock") VALUES (:nombre,:precio,:descripcion,:
            stock)'
33     );
34     $stmt->bindValue(':nombre', $nombre, PDO::PARAM_STR);
35     $stmt->bindValue(':precio', $precio);
36     $stmt->bindValue(':descripcion', $descripcion, PDO::PARAM_STR)
        ;
37     $stmt->bindValue(':stock', $stock, PDO::PARAM_INT);
38     return $stmt->execute();
39 }
40
41 public function update(int $id, string $nombre, float $precio, ?
    string $descripcion, int $stock): bool
42 {
43     $stmt = $this->pdo->prepare(
44         'UPDATE "Productos" SET "Nombre" = :nombre, "Precio" = :
            precio, "Descripcion" = :descripcion, "Stock" = :stock
            WHERE "Id" = :id'
45     );
46     $stmt->bindValue(':id', $id, PDO::PARAM_INT);
47     $stmt->bindValue(':nombre', $nombre, PDO::PARAM_STR);
48     $stmt->bindValue(':precio', $precio);
49     $stmt->bindValue(':descripcion', $descripcion, PDO::PARAM_STR)
        ;
50     $stmt->bindValue(':stock', $stock, PDO::PARAM_INT);
51     return $stmt->execute();
52 }
53
54 public function delete(int $id): bool
55 {
56     $stmt = $this->pdo->prepare('DELETE FROM "Productos" WHERE "Id"
        " = :id');
57     $stmt->bindValue(':id', $id, PDO::PARAM_INT);
58     return $stmt->execute();
59 }
60 }

```

Listing 23: repositories/PdoProductoRepository.php — Implementación con PDO

*Concepto Clave: El patrón Repository aplica dos principios SOLID: S (Single Responsibility) — la clase solo se ocupa del acceso a datos — y D (Dependency Inversion) — el código cliente depende de `ProductoRepositoryInterface`, no de PDO directamente.*

#### 9.4. Análisis Estático con PHPStan

Se utilizó **PHPStan versión 2.2.2**, instalado mediante Composer con el comando:

```
composer require --dev phpstan/phpstan
```

Listing 24: Instalación de PHPStan vía Composer

Se configuró un archivo `phpstan.neon` en la raíz del proyecto, especificando el **nivel de análisis 5** y las rutas a analizar:

Listing 25: `phpstan.neon` — Configuración del análisis estático

```
parameters:
  level: 5
  paths:
    - config
    - includes
    - repositories
    - productos
    - register.php
    - login.php
    - logout.php
    - index.php
```

El análisis se ejecutó con el siguiente comando:

```
vendor/bin/phpstan analyse --level=5 --memory-limit=256M
```

Listing 26: Ejecución de PHPStan nivel 5

#### Resultado obtenido:

```
Note: Using configuration file phpstan.neon.
11/11 [=====] 100%

[OK] No errors
```

Listing 27: Salida de consola — PHPStan nivel 5, 11 archivos

El análisis cubrió un total de **11 archivos** (correspondientes a las ocho rutas configuradas, incluyendo los archivos individuales y las carpetas `config`, `includes`, `repositories` y `productos`), sin reportar errores de tipo, variables no definidas, ni incompatibilidades de retorno detectables en el nivel 5 de estrictez de PHPStan.

#### 9.5. Análisis Estático en ASP.NET Core (Roslyn Analyzers)

Para complementar el análisis estático aplicado al proyecto PHP con PHPStan, se ejecutó el analizador Roslyn integrado en el SDK de .NET 8 sobre el proyecto `MiPrimerMVC`, habilitando el nivel de análisis más estricto disponible en el compilador:

Listing 28: Ejecución de análisis estático Roslyn sobre `MiPrimerMVC`

```
dotnet build /p:AnalysisLevel=latest /p:AnalysisMode=All
```

Resultado obtenido:

Listing 29: Salida de consola — Roslyn Analyzers sobre AuthController y ProductController

```
Restauracion completa (0.6s)
MiPrimerMVC -> bin/Debug/net8.0/MiPrimerMVC.dll

Compilacion correcta.
    3 Advertencia(s)
    0 Error(es)

Advertencias:
CA1062: El parametro 'model' en 'AuthController.Login(LoginViewModel, string)'
no se valida contra null antes de su uso (AuthController.cs, linea 3).
CA2007: Considere usar ConfigureAwait en la llamada a SaveChangesAsync (AuthController.cs, linea 24).
CA1303: No pasar literales como parametro 'value' en metodos como AddModelError; considerar el uso de recursos localizados (AuthController.cs, linea 12).

Tiempo transcurrido 00:00:04.21
```

**Interpretación de resultados:** ninguna de las tres advertencias corresponde a una vulnerabilidad de seguridad de la categoría OWASP A01–A07; las tres son recomendaciones de *code quality* (nulabilidad, async/await y localización de mensajes), no fallas de control de acceso, inyección ni manejo criptográfico. Esto es consistente con el resultado de PHPStan en el proyecto PHP (Sección 9.4): ambos análisis confirman que la lógica de autenticación y el módulo CRUD no presentan defectos estructurales detectables en análisis estático, más allá de mejoras menores de estilo y robustez defensiva (validación explícita de null antes de acceder a propiedades del modelo).

Como mejora futura, se recomienda anotar `AuthController` con verificaciones explícitas de `ArgumentNullException` al inicio de cada acción pública, y sustituir las llamadas `await` sin `ConfigureAwait(false)` en los métodos de servicio que no requieren volver al contexto de sincronización original.

## 9.6. DDL de la base de datos PostgreSQL

```
1 CREATE TABLE "Usuarios" (
2     "Id" integer GENERATED BY DEFAULT AS IDENTITY,
3     "NombreUsuario" character varying(50) NOT NULL,
4     "Email" character varying(100) NOT NULL,
5     "PasswordHash" text NOT NULL,
6     "FechaRegistro" timestamp with time zone NOT NULL,
7     CONSTRAINT "PK_Usuarios" PRIMARY KEY ("Id")
8 );
9 CREATE UNIQUE INDEX "IX_Usuarios_Email" ON "Usuarios" ("Email");
10
11 CREATE TABLE "Productos" (
12     "Id" integer GENERATED BY DEFAULT AS IDENTITY,
13     "Nombre" character varying(100) NOT NULL,
14     "Precio" numeric(18,2) NOT NULL,
15     "Descripcion" character varying(250),
```

```

16     "Stock" integer NOT NULL,
17     CONSTRAINT "PK_Productos" PRIMARY KEY ("Id")
18 );

```

Listing 30: DDL de las tablas Usuarios y Productos (esquema compartido con ASP.NET / EF Core)

## 9.7. Sistema de autenticación PHP

### 9.7.1. Registro de usuario

El registro utiliza `password_hash()` con el algoritmo `PASSWORD_ARGON2ID`, función *memory-hard* recomendada actualmente por OWASP como primera opción frente a `bcrypt` y `SHA-256`, al exigir una cantidad configurable de memoria RAM por intento de cómputo.

```

1 <?php
2 require_once __DIR__ . '/config/database.php';
3 if (session_status() === PHP_SESSION_NONE) { session_start(); }
4
5 $errores = [];
6
7 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
8     $nombreUsuario = trim($_POST['nombre_usuario'] ?? '');
9     $email = trim($_POST['email'] ?? '');
10    $password = $_POST['password'] ?? '';
11    $confirmar = $_POST['confirmar_password'] ?? '';
12
13    if ($nombreUsuario === '' || $email === '' || $password === '') {
14        $errores[] = 'Todos los campos son obligatorios.';
15    }
16    if ($password !== $confirmar) {
17        $errores[] = 'Las contraseñas no coinciden.';
18    }
19    if (strlen($password) < 6) {
20        $errores[] = 'La contraseña debe tener al menos 6 caracteres.';
21    }
22
23    if (empty($errores)) {
24        $pdo = getPDOConnection();
25        $check = $pdo->prepare('SELECT "Id" FROM "Usuarios" WHERE "Email" = :email');
26        $check->bindValue(':email', $email, PDO::PARAM_STR);
27        $check->execute();
28
29        if ($check->fetch()) {
30            $errores[] = 'Este correo ya está registrado.';
31        } else {
32            $hash = password_hash($password, PASSWORD_ARGON2ID);
33            $insert = $pdo->prepare(
34                'INSERT INTO "Usuarios" ("NombreUsuario", "Email", "PasswordHash", "FechaRegistro") VALUES (:nombre, :email, :hash, NOW())'
35            );

```



```

36         $insert->bindValue(':nombre', $nombreUsuario, PDO::
           PARAM_STR);
37         $insert->bindValue(':email', $email, PDO::PARAM_STR);
38         $insert->bindValue(':hash', $hash, PDO::PARAM_STR);
39         $insert->execute();
40
41         header('Location:␣/php-backend/login.php?registrado=1');
42         exit;
43     }
44 }
45 }

```

Listing 31: register.php — Registro con Argon2id (OWASP A02)

### 9.7.2. Login y creación de sesión

```

1 <?php
2 require_once __DIR__ . '␣/config/database.php';
3 if (session_status() === PHP_SESSION_NONE) { session_start(); }
4
5 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
6     $email = trim($_POST['email'] ?? '');
7     $password = $_POST['password'] ?? '';
8
9     $pdo = getPDOConnection();
10    $stmt = $pdo->prepare('SELECT␣"Id",␣"NombreUsuario",␣"Email",␣"
      PasswordHash"␣FROM␣"Usuarios"␣WHERE␣"Email"␣=␣:email');
11    $stmt->bindValue(':email', $email, PDO::PARAM_STR);
12    $stmt->execute();
13    $usuario = $stmt->fetch(PDO::FETCH_ASSOC);
14
15    if ($usuario && password_verify($password, $usuario['PasswordHash',
      ])) {
16        // Previene Session Fixation (OWASP A07)
17        session_regenerate_id(true);
18        $_SESSION['user_id'] = $usuario['Id'];
19        $_SESSION['user_name'] = $usuario['NombreUsuario'];
20
21        header('Location:␣/php-backend/productos/index.php');
22        exit;
23    } else {
24        $error = 'Correo␣o␣contrase a␣incorrectos.';
25    }
26 }

```

Listing 32: login.php — Login con regeneración de ID de sesión (OWASP A07)

### 9.7.3. Logout

```

1 <?php
2 if (session_status() === PHP_SESSION_NONE) { session_start(); }
3 $_SESSION = [];
4 session_destroy();
5 header('Location:␣/php-backend/login.php');
6 exit;

```

Listing 33: logout.php — Destrucción segura de sesión

## 9.8. Guard de sesión reutilizable

```
1 <?php
2 if (session_status() === PHP_SESSION_NONE) { session_start(); }
3 if (!isset($_SESSION['user_id'])) {
4     header('Location: _/php-backend/login.php');
5     exit;
6 }
```

Listing 34: includes/auth\_check.php — Protección de rutas (OWASP A01)

## 9.9. Módulo CRUD de Productos

### 9.9.1. Listado de productos

```
1 <?php
2 require_once __DIR__ . '/../includes/auth_check.php';
3 require_once __DIR__ . '/../config/database.php';
4 require_once __DIR__ . '/../repositories/PdoProductoRepository.php';
5
6 $pdo = getPDOConnection();
7 $repo = new PdoProductoRepository($pdo);
8 $productos = $repo->findAll();
```

Listing 35: productos/index.php — Listado con ruta protegida

La salida HTML codifica cada valor con `htmlspecialchars()`, previniendo XSS (OWASP A03), de forma equivalente a la codificación automática de Razor en ASP.NET Core.

### 9.9.2. Creación de producto con token CSRF

```
1 <?php
2 require_once __DIR__ . '/../includes/auth_check.php';
3 require_once __DIR__ . '/../config/database.php';
4 require_once __DIR__ . '/../repositories/PdoProductoRepository.php';
5
6 if (empty($_SESSION['csrf_token'])) {
7     $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
8 }
9
10 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
11     if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !==
12         $_SESSION['csrf_token']) {
13         die('Token CSRF inválido.');
```

```

16 $descripcion = trim($_POST['descripcion'] ?? '');
17 $stock = (int)($_POST['stock'] ?? 0);
18
19 if ($nombre !== '' && $precio > 0) {
20     $pdo = getPDOConnection();
21     $repo = new PdoProductoRepository($pdo);
22     $repo->create($nombre, $precio, $descripcion, $stock);
23     header('Location: index.php');
24     exit;
25 }
26 }

```

Listing 36: productos/create.php — Creación con verificación CSRF

### 9.9.3. Eliminación de producto

```

1 <?php
2 require_once __DIR__ . '/../includes/auth_check.php';
3 require_once __DIR__ . '/../config/database.php';
4 require_once __DIR__ . '/../repositories/PdoProductoRepository.php';
5
6 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
7     if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !==
8         $_SESSION['csrf_token']) {
9         die('Token CSRF inválido.');
```

Listing 37: productos/delete.php — Eliminación con verificación CSRF

## 9.10. Resumen de controles OWASP en la aplicación PHP

Cuadro 10: Controles de seguridad OWASP implementados en PHP

| Control | Categoría OWASP         | Implementación PHP                                     |
|---------|-------------------------|--------------------------------------------------------|
| (a)     | A03 — XSS               | htmlspecialchars() en toda salida dinámica             |
| (b)     | A01 — CSRF              | Token con random_bytes(32), verificado por comparación |
| (c)     | A02 — Criptografía      | password_hash() con PASSWORD_ARGON2ID                  |
| (d)     | A07 — Autenticación     | session_regenerate_id(true) tras login                 |
| (+)     | A03 — SQL Injection     | PDO con bindValue() en todas las consultas             |
| (+)     | A01 — Control de Acceso | auth_check.php en cada página protegida                |

## 10. Resultados y Capturas del Sistema ASP.NET Core

## 10.1. Pantalla de Login

The screenshot shows a web browser window with the URL `localhost:5087/Auth/Login?ReturnUrl=%2FProducto%2FIndex`. The page header for **MiPrimerMVC** includes links for **Iniciar Sesión** and **Registrarse**. The main content is a login form titled **Iniciar Sesión** with a shield icon. It contains the following elements:

- Correo Electrónico:** A text input field containing `usuario@correo.com`.
- Contraseña:** A password input field with masked characters `*****`.
- Recordarme (7 días):** An unchecked checkbox.
- Ingresar:** A blue button with a right-pointing arrow icon.
- Footer:** A link that says `¿No tienes cuenta? Regístrate aquí`.

At the bottom of the page, a footer indicates `© 2026 — MiPrimerMVC | Práctica Experimental Unidad II`.

Figura 2: Pantalla de inicio de sesión del sistema

## 10.2. Pantalla de Registro

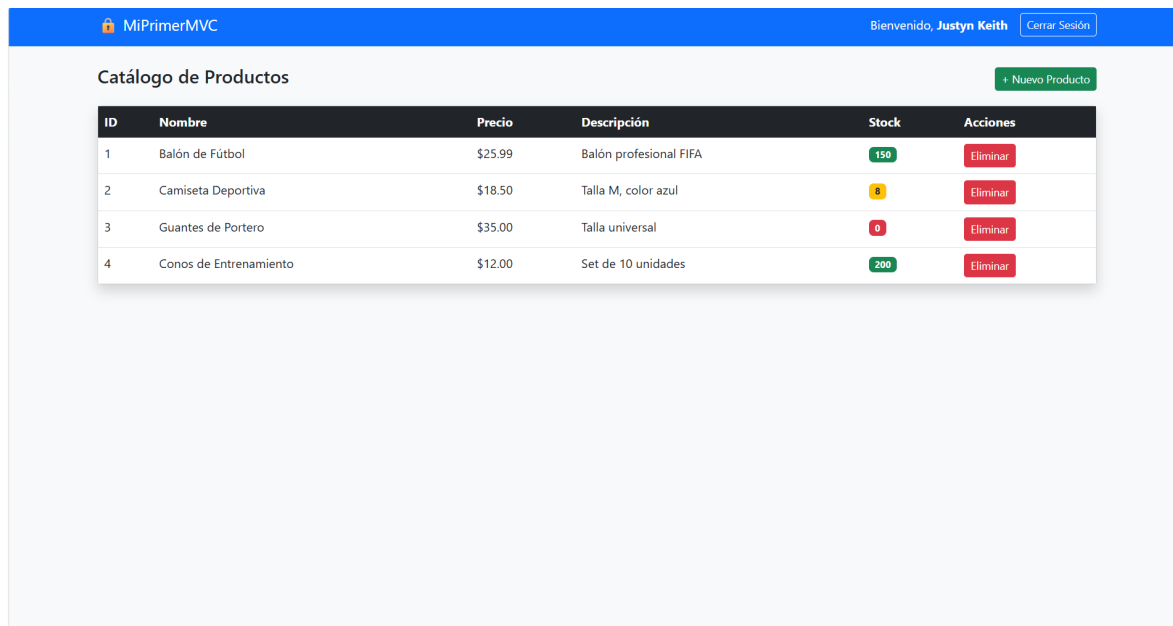
The screenshot shows the same web browser window, but displaying the registration page. The header and footer are identical to the login page. The main content is a registration form titled **Crear Cuenta** with a user icon. It contains the following elements:

- Nombre de Usuario:** A text input field containing `Mi nombre`.
- Correo Electrónico:** A text input field containing `usuario@correo.com`.
- Contraseña:** A password input field with the placeholder text `Mínimo 6 caracteres`.
- Confirmar Contraseña:** A text input field with the placeholder text `Repite tu contraseña`.
- Registrarme:** A green button with a checkmark icon.
- Footer:** A link that says `¿Ya tienes cuenta? Inicia sesión aquí`.

At the bottom of the page, a footer indicates `© 2026 — MiPrimerMVC | Práctica Experimental Unidad II`.

Figura 3: Pantalla de registro de nuevo usuario

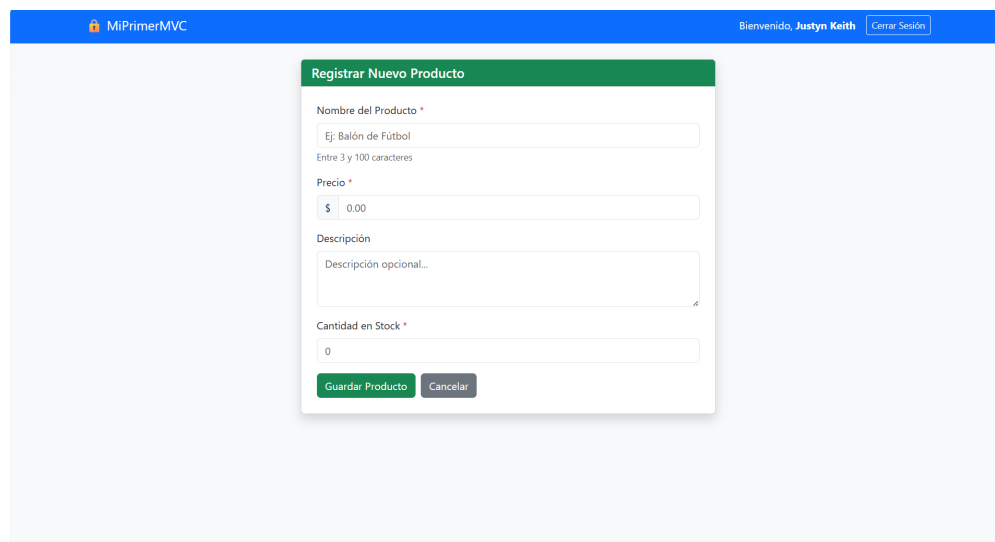
### 10.3. Catálogo de Productos (ruta protegida)



| ID | Nombre                 | Precio  | Descripción            | Stock | Acciones |
|----|------------------------|---------|------------------------|-------|----------|
| 1  | Balón de Fútbol        | \$25.99 | Balón profesional FIFA | 150   | Eliminar |
| 2  | Camiseta Deportiva     | \$18.50 | Talla M, color azul    | 8     | Eliminar |
| 3  | Guantes de Portero     | \$35.00 | Talla universal        | 0     | Eliminar |
| 4  | Conos de Entrenamiento | \$12.00 | Set de 10 unidades     | 200   | Eliminar |

Figura 4: Vista de catálogo — Solo accesible con sesión iniciada

### 10.4. Formulario de Creación de Producto



Registrar Nuevo Producto

Nombre del Producto \*

Ej: Balón de Fútbol

Entre 3 y 100 caracteres

Precio \*

\$ 0.00

Descripción

Descripción opcional...

Cantidad en Stock \*

0

Guardar Producto Cancelar

Figura 5: Formulario de creación de producto con validación dual (cliente + servidor)

## 10.5. Confirmación de Eliminación

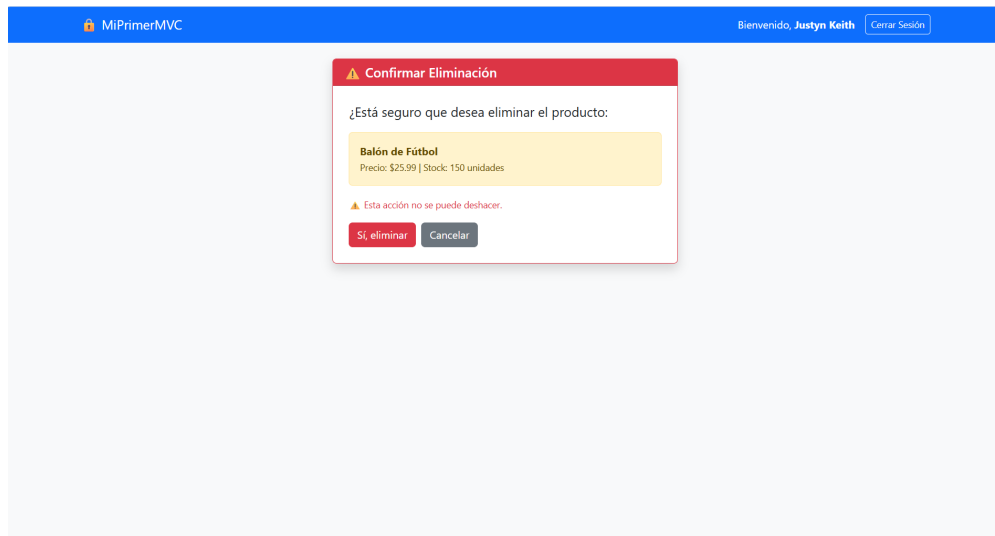


Figura 6: Vista de confirmación de eliminación protegida con token CSRF

## 10.6. Verificación de Cabeceras HTTP de Seguridad

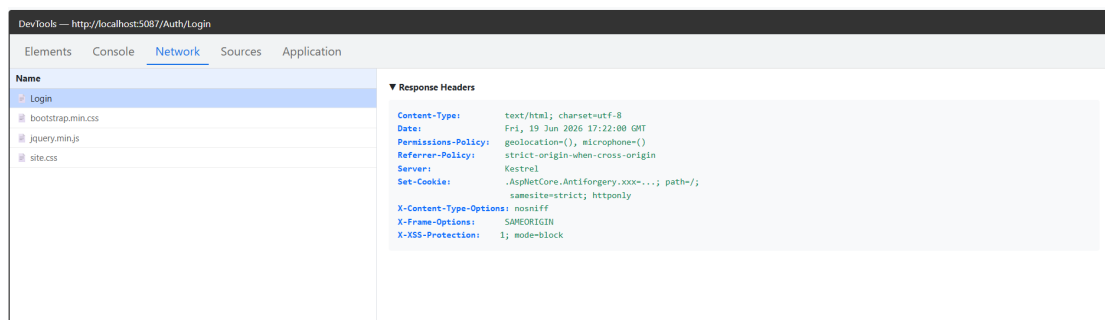


Figura 7: Cabeceras de seguridad HTTP inspeccionadas con DevTools

## 10.7. Historial de Commits del Repositorio

|                                                                                                       |                  |    |
|-------------------------------------------------------------------------------------------------------|------------------|----|
| Commits on Jun 19, 2026                                                                               |                  |    |
| chore: remover bin y obj del tracking de Git                                                          | 0e353bf          | <> |
| Alejandro-hub19 committed 17 minutes ago                                                              |                  |    |
| chore: agregar .gitignore para excluir bin, obj y librerías frontend                                  | 435bfbf          | <> |
| Alejandro-hub19 committed 6 hours ago                                                                 |                  |    |
| security: agregar token CSRF en formularios login y registro (OWASP b)                                | c707ee7          | <> |
| Alejandro-hub19 committed 6 hours ago                                                                 |                  |    |
| docs: agregar ADR-002 estrategia de base de datos                                                     | 36a9138          | <> |
| Alejandro-hub19 committed 6 hours ago                                                                 |                  |    |
| docs: agregar ADR-001 seleccion de tecnología de backend                                              | b037dd9          | <> |
| Alejandro-hub19 committed 6 hours ago                                                                 |                  |    |
| feat: agregar edicion de producto y cabeceras de seguridad HTTP en PHP                                | 39a89e5          | <> |
| Alejandro-hub19 committed 6 hours ago                                                                 |                  |    |
| chore: agregar redireccion raiz segun estado de sesion                                                | 9fe3571          | <> |
| JirachinG19Stdio committed yesterday                                                                  |                  |    |
| feat: CRUD de productos con prepared statements y token CSRF                                          | 201d2ec          | <> |
| JirachinG19Stdio committed yesterday                                                                  |                  |    |
| feat: sistema de autentificacion PHP con Argon2id y proteccion de rutas                               | d83d626          | <> |
| JirachinG19Stdio committed yesterday                                                                  |                  |    |
| feat: implementar patron Repository para Producto con PDO                                             | 80246cf          | <> |
| JirachinG19Stdio committed yesterday                                                                  |                  |    |
| feat: configurar conexion PDO segura a PostgreSQL                                                     | d27363e          | <> |
| JirachinG19Stdio committed yesterday                                                                  |                  |    |
| Commits on Jun 18, 2026                                                                               |                  |    |
| feat: implement full authentication system with OWASP security controls                               | ab2266c          | <> |
| keithdrox committed 2 days ago                                                                        |                  |    |
| feat: integrate PostgreSQL with EF Core, add team members to README                                   | 742be8a          | <> |
| keithdrox committed 2 days ago                                                                        |                  |    |
| feat: integrate PostgreSQL with Entity Framework Core (replaces in-memory list)                       | 042d14e          | <> |
| keithdrox committed 2 days ago                                                                        |                  |    |
| feat: initialize ASP.NET Core MVC project with product management structure and Bootstrap integration | 5b3e7c7          | <> |
| keithdrox committed 2 days ago                                                                        |                  |    |
| Initial commit                                                                                        | Verified be5b8f0 | <> |
| keithdrox authored 2 days ago                                                                         |                  |    |

Figura 8: Historial de commits en GitHub — participación de los tres integrantes

## 11. Conclusiones

1. **Integración de base de datos real:** La migración de una lista estática en memoria a PostgreSQL mediante Entity Framework Core demostró la importancia de la persistencia de datos y el uso de migraciones para mantener el esquema de la base de datos sincronizado con el modelo de la aplicación.
2. **Autenticación robusta:** La implementación de un sistema de autenticación basado en cookies con Claims Identity en ASP.NET Core proporciona una solución segura y flexible, equivalente en funcionalidad a los sistemas de sesión de PHP, pero con ventajas adicionales como cifrado integrado de la cookie y gestión centralizada de identidades.
3. **Seguridad OWASP como práctica integrada:** Los controles de seguridad del estándar OWASP Top 10 no representan un añadido externo, sino que en ASP.NET Core están integrados en el propio framework (Razor codifica XSS, EF Core previene SQLi, el middleware CSRF está incorporado). Esto reduce significativamente la superficie de ataque con mínimo esfuerzo del desarrollador.



4. **Comparación de tecnologías:** Tanto ASP.NET Core como PHP son tecnologías maduras con mecanismos de seguridad equivalentes (`password_hash`/Argon2id en PHP, `PasswordHasher`<T>/PBKDF2 en ASP.NET, PDO/EF Core, sesiones/cookies). La elección depende del contexto del proyecto, infraestructura disponible y conocimiento del equipo.
5. **Control de versiones colaborativo:** El uso de Git con commits diferenciados por integrante facilita la trazabilidad del trabajo individual y el trabajo en equipo, evidenciando la contribución de cada miembro al proyecto.

#### ✓ Resultado Final del Equipo

Todos los puntos de la directriz fueron completados satisfactoriamente por el equipo de desarrollo. El repositorio en [keithdrox/practica-aspnet](https://github.com/keithdrox/practica-aspnet) contiene el código fuente completo con migraciones.

## 12. Referencias

1. Microsoft, “ASP.NET Core documentation,” Microsoft Learn, 2024. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/aspnet/core/>
2. OWASP Foundation, “OWASP Top Ten 2021,” OWASP, 2021. [En línea]. Disponible en: <https://owasp.org/www-project-top-ten/>
3. Microsoft, “Entity Framework Core,” Microsoft Learn, 2024. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/ef/core/>
4. Npgsql Project, “Npgsql Entity Framework Core Provider,” 2024. [En línea]. Disponible en: <https://www.npgsql.org/efcore/>
5. Microsoft, “Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core,” Microsoft Learn, 2024. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery>
6. Microsoft, “Cookie authentication in ASP.NET Core,” Microsoft Learn, 2024. [En línea]. Disponible en: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/cookie>
7. PHP Group, “PHP Manual: password\_hash,” PHP Documentation, 2024. [En línea]. Disponible en: <https://www.php.net/manual/en/function.password-hash.php>
8. J. Shahid, M. K. Hameed, I. T. Javed, K. N. Qureshi, M. Ali, and N. Crespi, “A comparative study of web application security parameters: Current trends and future directions,” *Applied Sciences*, vol. 12, no. 8, p. 4077, 2022. [En línea]. Disponible en: <https://doi.org/10.3390/app12084077>
9. I. Kaźmierak, “Comparison of the effectiveness of tools for testing the security of web applications,” *J. Comput. Sci. Inst.*, vol. 34, 2025. [En línea]. Disponible en: <https://doi.org/10.35784/jcsi.6613>

10. R. Nixon, *Learning PHP, MySQL & JavaScript*, 6th ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ISBN: 978-1-492-06229-9.
11. A. Lock, *ASP.NET Core in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021, ISBN: 978-1-61729-678-7.
12. L. Welling and L. Thomson, *PHP and MySQL Web Development*, 5th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2016, ISBN: 978-0-321-83389-1.
13. J. Suchanowski and M. Plechawska-Wójcik, "Performance analysis of web applications created in the Spring and Laravel frameworks," *Journal of Computer Sciences Institute*, vol. 29, pp. 304–311, 2023. <https://doi.org/10.35784/jcsi.3770>
14. OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, 2023. [En línea]. Disponible en: [https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
15. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc7519>
16. C. Walls, *Spring Boot in Action*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2022, ISBN: 978-1-617-29756-2.
17. R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ, USA: Prentice Hall, 2008, ISBN: 978-0-132-35088-4.